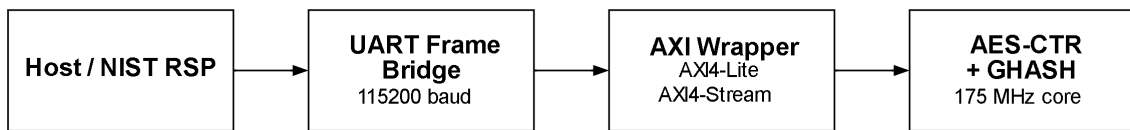


FPGA3 AES-GCM IP

175 MHz FPGA 設計與驗證報告

需求追蹤、Scoreboard、Signoff 與實板證據



UART 115200 baud → AXI4-Lite / 8-bit AXI4-Stream → 175 MHz crypto core

目標 FPGA： Digilent Arty A7-100T, XC7A100TCSG324-1

核心時脈： 175 MHz (週期 5.714 ns)

工具版本： Xilinx Vivado 2021.1, build 3247384

Git branch： `verify/sva-uvm-board`

Build-base commit： `ece78ce9677872d0d20da3b785ab83a48b6a8492`

Source/verification commit： `4246ebc7a72ce8eb1a49d56969aae6e151578c11`

報告日期： 2026-08-23

結論：175 MHz static、dynamic 與實板驗證皆通過；整合狀態僅因 formal scope 為 PARTIAL PASS，PartPin 維持 NO / NOT RUN。

本報告所有數值均取自目前工作區 RTL、Vivado DCP/RPT、XSIM log 與 COM4 實板證據；未量測項目會明確標示。

摘要

本設計完成一個支援 AES-128、AES-192 與 AES-256 的 AES-GCM authenticated-encryption IP，並以 AXI4-Lite 設定介面、8-bit AXI4-Stream 資料介面與 Arty A7 UART 測試 wrapper 整合。核心以 AES counter mode 產生 keystream，以 GHASH 在 $GF(2^{128})$ 執行驗證；解密 plaintext 在 tag 驗證成功以前不對外釋放，並具備同步 ZEROIZE 與 protocol-abort 清除機制。

本報告把 signoff 分成四個互不代替的邊界：175 MHz full-board implementation 為 **PASS**，RTL/UVM/NIST dynamic verification 為 **PASS**，FPGA programming、六模式 smoke 與 47,250 筆 hardware NIST 為 **PASS**；formal verification 為 **PARTIAL PASS**，因為目前只完成 19 個 assumption-free combinational proofs，尚未完成 sequential 與 whole-design formal closure。整合 run_all 為 5 個 suite PASS、1 個 formal suite PARTIAL PASS、0 NOT RUN、0 FAIL。Standalone PartPin 為 **NO / NOT RUN**；200 MHz 不在本版範圍，也未執行。

最終板級設計在 175 MHz 完成 fresh synthesis、placement、physical optimization 與 routing，並由 immutable routed DCP 進行獨立 v8c strict audit。Routed signoff 為 WNS +0.029 ns、TNS 0、setup failing endpoints 0、WHS +0.044 ns、THS 0、hold failing endpoints 0；pulse-width WPWS +1.607 ns、TPWS 0、failing endpoints 0。17,758 個 routable nets 全部完成 routing，route error、DRC Error/Critical Warning、methodology Error/Critical Warning 與 unsafe CDC category 均為 0。另有 2 個一般 SYNTH-6 Warning，因此不宣稱 warning-free。

XSim UVM 1.2 以 seeds 175001、175019、175039 共重建並檢查 24 筆 transactions，scoreboard failure 與 UVM error/fatal 均為 0，每個 seed 的 maintained functional coverage 為 100%。完整 FPGA NIST 在 COM4、115200 baud 執行 47,250/47,250 pass、fail 0，其中包含 11,908 筆預期 authentication failure；status/data/tag violations 全為 0，並完成第二次獨立 evidence reconciliation。

吞吐率測試以 100 筆、每筆 1024-bit payload 計算：175 MHz 下 AES-128/192/256 加密皆為 1.065398 Gbit/s，解密皆為 1.051705 Gbit/s。v8a bitstream 已經 Vivado Hardware Manager 程式化到 xc7a100t_0，六模式 smoke 為 6 pass、0 fail。上述 UVM 100% 不是 code/assertion/exhaustive coverage；0.461 W 是 vectorless estimate，不是實板量測；39.098 cases/s 是 UART harness 速率，不是 1 Gbit/s crypto throughput。

語音術語對照：本報告以下採 AES 標準名稱 *SubBytes*、*ShiftRows*、*MixColumns*、*AddRoundKey*、*Key Expansion*。本設計沒有使用 Edward Lorenz 的 Lorenz chaotic system；若口述的「Edward Lorenz」原意是 *AddRoundKey*，其原理即為 state 與 128-bit round key 的逐位 XOR。

目录

摘要	i
第一章 設計範圍與驗證結論	1
1.1 本報告涵蓋的實體	1
1.2 最終狀態總表	2
1.3 需要正確解讀的限制	3
第二章 IC / FPGA 設計流程與專案方法	4
2.1 從規格到實板的完整流程	4
2.2 本 AES-GCM 專案如何落實各階段	5
2.2.1 規格、演算法與微架構	5
2.2.2 RTL 與 clock-domain 規則	6
2.2.3 Synthesis 與 technology mapping	6
2.2.4 Implementation、timing closure 與 signoff	6
2.3 Bitstream、實板驗證與 release evidence	7
2.4 FPGA flow 與 ASIC flow 的界線	7
第三章 AES 與 GCM 演算法原理	8
3.1 AES state、key size 與 round 數	8
3.2 SubBytes：有限體反元素與 affine transform	8
3.3 ShiftRows：固定 byte permutation	9
3.4 MixColumns： $GF(2^8)$ 固定矩陣乘法	9
3.5 AddRoundKey：128-bit XOR	9
3.6 Key Expansion：RotWord、SubWord、Rcon	9
3.7 GCM：CTR confidentiality 與 GHASH authentication	10
3.8 目前 GHASH 的 16-bit digit-serial 架構	10
第四章 RTL 硬體架構與平行化	11
4.1 整體資料流	11

4.2	AES engines：三 context pipeline 加兩個 first-block engines	11
4.3	兩筆 sensitive data queue 與 throughput 改善	12
4.4	FIFO 與 backpressure	12
4.5	ZEROIZE 與 security locality	13
第五章	Clock、cycle 與吞吐率	14
5.1	175 MHz 時脈產生	14
5.2	吞吐率公式與測試定義	14
5.3	Latency 與 throughput 的差異	16
5.4	實板 UART throughput	16
第六章	Synthesis、資源使用率與功率	18
6.1	Synthesis 的目的	18
6.2	板級 routed 資源	18
6.3	主要 hierarchy 資源分布	19
6.4	使用率降低：UART wide-register arrays 改成 byte memories	19
6.5	Placement 分布與 routing 可視化	21
6.6	功率估算	23
第七章	Implementation 與 timing closure	25
7.1	Vivado flow 與 Tcl 的角色	25
7.2	Timing iteration 結果	26
7.3	175 MHz final scope 與 throughput 補償	26
7.4	Final route 與 DRC	28
第八章	Timing 問題、同步化選擇與演算法 / 微架構改善	33
8.1	同步還是非同步：本設計的實際選擇	33
8.2	Timing 問題、根因與收斂過程	34
8.3	實際採用的 Timing 解法	35
8.4	演算法等價實作與硬體微架構改善	35
8.5	吞吐率改善、面積與 security 取捨	36
8.6	最終驗證與可重現性界線	37
第九章	Wrapper、UART 封包與傳輸效率	38
9.1	Production AXI IP wrapper：設計目標與責任邊界	38
9.1.1	AXI4-Lite control plane 與 register map	39
9.1.2	三條命令不是同一組 admission checks	40

9.1.3	Input plane : 2-entry registered-head queue 與 frame credits	40
9.1.4	Output plane : elasticity 、outermost frame ownership 與 mode-dependent ordering	41
9.1.5	ZEROIZE : emergency flush , 而不是 graceful drain	42
9.1.6	Protocol abort 與 ZEROIZE 的差異	43
9.1.7	設計取舍、可驗證性與報告結論	43
9.2	UART board wrapper	44
9.3	Protocol efficiency 與錯誤處理	44
第十章	驗證策略、NIST 與 FPGA 實板結果	46
10.1	端到端設計與驗證流程	46
10.1.1	Stage-gated verification plan	46
10.1.2	RTL 模擬 gate : UVM、NIST 與 SVA 的先後關係	47
10.1.3	UVM 如何從訊號走到 scoreboard	48
10.1.4	SVA、UVM、Vivado 與實板 checker 的分工	51
10.1.5	可重現的執行入口	52
10.2	Verification plan 、oracle 與 comparison chain	53
10.3	RTL regressions	54
10.4	SVA : cycle-by-cycle temporal checker	55
10.5	UVM、coverage、formal 與 integrated replay	57
10.6	Positive、negative tests 與未覆蓋範圍	58
10.7	FPGA programming 狀態	58
10.8	六模式 smoke 與 47,250 筆 NIST RSP final result	59
第十一章	遇到的問題、根因與解法	61
第十二章	工具、Tcl 驅動方式與檔案位置	63
12.1	本次使用的工具	63
12.2	如何驅動 Tcl	64
12.3	RTL、SVA 與 wrapper 位置	64
12.4	Final implementation、圖片與證據位置	65
第十三章	結論與後續工作	68
附录 A	Artifact identity 與 hashes	69
附錄 B	參考資料	70

插图

3.1	目前 RTL 的 16-bit digit-serial GHASH 資料路徑	10
4.1	AES-GCM core 的主要資料流、queues 與 crypto engines	11
5.1	Vivado 2021.1 原生 clock network 與 clock interaction 視圖。	15
5.2	175 MHz 下的六模式 RTL throughput ; 三種 key size 皆大於 1 Gbit/s	16
6.1	LUT、FF、BRAM 與 DSP 使用率圖 ; 數據來自 final routed DCP	19
6.2	Synthesis 後的原生資源使用率與 critical-path netlist 視圖。	20
6.3	Final routed DCP 的 active clock-region placement 摘要。圖中 FF 為該 clock 的實際 clock loads ; LUTM/RAMB 欄位取自 Vivado clock-utilization report , 並非全器件 primitive 總數。	21
6.4	Vivado 原生 implementation placement 視圖。	22
6.5	Final routed vectorless power breakdown	23
6.6	Vivado 原生 Power Summary : 總 on-chip power 為 0.461 W , 其中 dynamic 0.363 W、device static 0.099 W ; confidence level 為 Medium。	24
7.1	Board wrapper 從 wide registers 到 final route 的 timing 收斂	26
7.2	Vivado 原生 post-route timing signoff 視圖。	27
7.3	Vivado write_schematic 匯出的 post-route setup / hold critical paths。	29
7.4	Worst setup path 的邏輯結構與實體 delay 證據。	30
7.5	Vivado 原生 routing resources 與 congestion analysis。	31
7.6	Vivado 原生 Run DRC 結果 : No Violations Found。	32
9.1	Vivado 原生 Package / I/O 視圖的清晰裁切 ; 顯示 XC7A100T CSG324 package 的電源、時脈與 I/O pins , 供 board-level pin planning 查核。DRC 的 No Violations Found 證據另見圖 7.6。	45
10.1	Full FPGA NIST source coverage 與 host-observed latency。左圖六個 RSP source 各執行並通過 7,875 筆 ; 右圖呈現 min、mean、median、p95、p99 與 max。這不是 AES-GCM core cycle latency。	60

表格

1.1	最終 FPGA signoff 狀態	2
2.1	本專案的 IC / FPGA stage-gated 設計流程	5
3.1	AES key size 與 round 數	8
4.1	核心 crypto unit 的 clock latency 與 initiation interval	12
6.1	Arty A7-100T routed resource utilization	18
6.2	Final routed utilization by hierarchy	19
6.3	UART bridge memory refactor 前後資源與 synth timing	21
6.4	Vivado routed vectorless power estimate	23
7.1	Timing iteration 詳細數據	26
7.2	Route、DRC 與 methodology 摘要	28
8.1	同步 / 非同步邊界與處理方式	33
8.2	Timing 問題從症狀到 final closure 的工程鏈	34
8.3	演算法等價實作與微架構改善	36
8.4	175 MHz 吞吐率改善前後比較	37
9.1	Production AXI wrapper 的三個責任平面	38
9.2	Production AES-GCM AXI wrapper 的精確 AXI4-Lite register map	39
9.3	三條公開 output 的實際 buffering 與 timing 特性	41
9.4	合法完成與錯誤路徑的公開輸出順序	42
10.1	從規格、RTL 到 FPGA 實板的端到端驗證流程	46
10.2	Simulation domain 內三條獨立 checker path ; full RSP path 為 <code>tb_nist + bound</code> SVA、UVM regression 另行執行。	48
10.3	UVM transaction 從 stimulus、interface handshake、passive observation 到 score- board 的完整生命週期。	49

10.4 XSim UVM 1.2 驗證環境的 source/class 分工；UVM library 提供共通 framework， 本專案實作 AES-GCM/AXI 驗證語意。	50
10.5 SVA 從編譯、bind/elaboration、逐拍監看到 runner hard gate 的執行流程。	52
10.6 Requirement-to-checker traceability matrix	54
10.7 SVA checker 分組與檢查目的	55
10.8 SVA evidence ledger 與可宣稱的範圍	56
10.9 FPGA hardware verification final status	59
11.1 Verification failure triage：症狀、修正、oracle 與證據	61
12.1 工具與用途	63
12.2 主要 RTL 與 verification source files	64
A.1 Final artifact SHA-256	69

第一章 設計範圍與驗證結論

1.1 本報告涵蓋的實體

本報告同時區分三個層級，避免把核心、wrapper 與板級傳輸混為一談：

1. **AES-GCM 核心**：AES round engines、key expansion、GHASH、descriptor/data queues、authentication 與 ZEROIZE。
2. **AXI integration wrapper**：`aes_gcm_axi_top.v`，以 AXI4-Lite 控制暫存器與 AXI4-Stream byte interfaces 包裝核心。
3. **Arty board test wrapper**：`arty_a7_100t_aes_gcm_uart_rsp_top.v`，加入 MMCM、2-stage ASYNC_REG run-request synchronizer、4-stage synchronous reset-release qualifier、UART RX/TX 與 NIST RSP framing bridge。

1.2 最終狀態總表

表 1.1: 最終 FPGA signoff 狀態

項目	結果	證據 / 說明
核心時脈	175 MHz	MMCM: $100 \times 7/4 = 175$ MHz ; 週期 5.714 ns
Synthesis	完成	v8 fresh build ; WNS -0.298 ns 、 TNS -11.728 ns 、 setup FEP 292
Placement + phys_opt	完成	setup WNS +0.075 ns ; hold WHS -0.082 ns 、 THS -0.364 ns 、 FEP 9
Routing	PASS	audited WNS +0.029 ns 、 WHS +0.044 ns 、 TNS/TH-S/FEP 全為 0
Pulse width	PASS	WPWS +1.607 ns 、 TPWS 0 、 failing endpoints 0
Routing legality	PASS	17,758/17,758 routable nets fully routed 、 routing errors 0
DRC / methodology	PASS	DRC violations 0 ; methodology Error/Critical Warning 0 、 一般 Warning 2
CDC / reset	PASS	2 個 CDC-3 safe synchronizers ; 2-stage request sync + 4-stage release qualifier
check_timing	PASS	10 個 critical categories 為 0 ; false-path allowlists 為 input 2 / output 5
Bitstream / config image	PASS	v8c strict-audit .bin 與 hardware-tested v8a .bin 完全相同 ; 完整 .bit 檔案 hashes 不同
FPGA programming	PASS	program_board.tcl 以 exact v8a .bit 回報 status=PASS ; device xc7a100t_0
RTL throughput	PASS	1.065398 Gbit/s encrypt ; 1.051705 Gbit/s decrypt
FPGA six-mode smoke	PASS	AES-128/192/256 encrypt/decrypt ; 6/6 pass
NIST FPGA test	PASS	47,250/47,250 pass 、 fail 0 、 first failure null ; 第二位獨立 reconciliation 亦通過
UVM / formal	部分完成	XSim UVM 24/0 ; open-source combinational formal 19/0 、 不代表 full formal closure
ModelSim UVM 1.2	PASS	Starter deterministic smoke 1/0 ; constrained random NOT RUN 、 coverage NOT COLLECTED
Integrated run_all	PARTIAL PASS	5 PASS 、 1 formal PARTIAL PASS 、 0 NOT RUN 、 0 FAIL
PartPin	NO	無 approved identity-matched production map ; 未執行 standalone replay

1.3 需要正確解讀的限制

- Routed setup margin 為 +0.029 ns ; 若換 Vivado build 、 constraints 、 RTL hash 或 implementation directive , 必須重跑 signoff 。
- 0.461 W 為 routed vectorless power estimate , 內部 switching activity 信心等級 Medium ; 它不是電源軌實測值 。
- 1.05–1.07 Gbit/s 是 RTL/AXI testbench 的 crypto payload throughput 。 UART 使用 115200 baud , 兩者是不同觀測邊界 。
- Methodology 報告含 2 個 SYNTH-6 Warning ; 結果是 Error/Critical-Warning clean , 不是 warning-free 。
- FPGA NIST 47,250 筆已經 exact-bitstream 綁定及兩次 reconciliation 通過 ; 它證明 UART wrapper 的向量正確性 , 但 39.098 cases/s 仍是 host/UART 交易速率 , 不是 crypto datapath throughput 。
- UVM 的 100% 是維護中 scenario bins 的 functional coverage ; 目前沒有 code coverage 、 assertion coverage 或 exhaustive protocol/security coverage 。
- Formal 結果只涵蓋 17 個 GHASH shift-power 與 2 個 AES S-box combinational assertions ; sequential/whole-design formal closure 尚未完成 。
- PartPin 尚未完成 ; full-board route 不能取代 standalone PartPin replay 。

第二章 IC / FPGA 設計流程與專案方法

2.1 從規格到實板的完整流程

本專案採用的是以 **stage gate 驗證的 FPGA 數位 IC 設計流程**。流程不是寫完 RTL 就直接燒錄，而是讓每一階段都有明確的輸入、輸出、判定條件與可追溯證據：

設計規格 → 演算法 / 微架構 → RTL → 前端驗證 → Synthesis → Implementation → Timing / Legality Signoff → Bitstream → FPGA 實板驗證 → Release Evidence
--

這條主線同時保留回饋迴路。功能錯誤回到 RTL 或 checker；資源超標回到資料路徑與記憶體映射；setup timing 失敗回到邏輯深度、fanout、pipeline 與 placement locality；CDC 或 reset 失敗回到同步器及 constraints；實板錯誤則依序核對 bitstream identity、wrapper、UART protocol 與演算法結果。只有前一個 gate 通過，結果才有資格進入下一階段。

表 2.1: 本專案的 IC / FPGA stage-gated 設計流程

階段	主要工作 / 決策	產出與證據	通過條件
1. 規格	定義 AES-128/192/256、encrypt/decrypt、175 MHz、> 1 Gbit/s、AXI/UART、ZEROIZE 與 authentication-before-release	設計需求、介面 / cycle 定義、security invariants、驗證計畫	功能、效能、安全與觀測邊界均可量測
2. 演算法與微架構	將 AES、CTR、GHASH 映射成可綜合硬體；選擇 context 數、FIFO 深度、GHASH digit size 與 back-pressure 策略	block architecture、latency / II 預估、面積與 timing 風險	數學等價，且 throughput 預算可達標
3. RTL	以同步時序設計 datapath、control、queues、ZEROIZE、AXI 與 board wrapper	Verilog RTL、constraints、SVA bind 與 testbench hooks	compile/elaboration clean、無 latch / multiple-driver
4. 前端驗證	directed smoke、UVM constrained-random、SVA cycle checks、NIST oracle comparison 與有限 formal proof	log、seed、coverage、assertion 與 mismatch evidence	scoreboard 0 mismatch、SVA 0 failure、必要 scenarios 命中
5. Synthesis	展平 / 重建 hierarchy、Boolean optimization、technology mapping、將 RTL 映射成 LUT、FF、BRAM 與 clock resources	synthesized DCP、utilization、control sets、synth timing	無非法網表；資源與 critical families 可接受
6. Implementation	opt\design、placement、physical optimization、routing 與 post-route optimization	placed / routed DCP、route status、timing paths、physical views	全部 nets routed、setup / hold 可收斂
7. Signoff	稽核 clocks、exceptions、CDC、pulse width、DRC、methodology、check_timing 與 hashes	audited reports、DCP / bitstream identity、hard-gate summary	WNS/WHIS ≥ 0；TNS/TH-S/FEP = 0；legality gates clean
8. 實板驗證	產生 / 燒錄 exact bitstream；經 UART 執行六模式 smoke 與 NIST RSP replay	programming log、board transcript、47,250-case summary	6/6 modes 及 47,250/47,250 vectors 通過
9. Release	固定 RTL、Tcl、reports、bitstream、hashes 與限制說明	Git commit、artifact manifest、可重現命令	證據可由同一版本重建與核對

2.2 本 AES-GCM 專案如何落實各階段

2.2.1 規格、演算法與微架構

演算法規格以 AES 與 GCM 的標準運算為基準：AES datapath 執行 SubBytes、ShiftRows、MixColumns、AddRoundKey 與 Key Expansion；GCM 以 counter mode 產生 keystream，並以 $GF(2^{128})$ 的 GHASH 驗證 AAD 與 ciphertext。硬體層不改變演算法結果，而是改變運算的排程、平行度、buffering 與資料重用方式。

為達成 175 MHz 下超過 1 Gbit/s 的核心吞吐率，微架構採三個共享 AES context、兩個 first-

block engines、16-entry keystream FIFO、兩筆 sensitive-data queue 與 16-bit digit-serial GHASH。這些結構讓不同 block 的 AES round 能交錯執行，並降低 AES、GHASH、input/output backpressure 彼此等待的比例。安全規格則要求 raw ZEROIZE 能在同一有效 edge 清除敏感狀態、驗證失敗前不得釋出未認證 plaintext，且 queue clear 後 stale head 不得 fire。

2.2.2 RTL 與 clock-domain 規則

核心資料路徑採**單一 175 MHz 同步 clock domain**；pipeline、FSM、FIFO 與 ZEROIZE priority 都在 clock edge 上更新。非同步方法只用在真正跨 clock/domain 的邊界：例如板級 run request 使用 2-stage ASYNC_REG synchronizer，reset deassertion 使用 4-stage synchronous release qualifier。這個分工避免在 AES/GHASH datapath 內加入不可控的非同步 handshake，也讓 STA 能完整分析 register-to-register paths。

RTL 完成後先做 compile/elaboration，檢查未宣告訊號、width、latch 與 multiple-driver；再用 directed tests 重現 reset、key/IV/AAD/data、backpressure、ZEROIZE、authentication failure 與 queue corner cases。UVM 負責 transaction generation、driver、monitor、scoreboard 與 functional coverage，SVA 則對每一個 clock edge 的 protocol/security invariant 做局部檢查。詳細 signal-to-scoreboard 與 property 分工列於第 10.1 節。

2.2.3 Synthesis 與 technology mapping

Synthesis 的工作是把行為 RTL 轉為 FPGA primitive netlist，並回答三個問題：邏輯是否可合法實現、資源是否放得下、尚未 placement 前的 timing 風險在哪裡。Vivado 會把組合運算映射成 LUT，把狀態映射成 FF，把合適的 memories/FIFOs 映射成 distributed RAM 或 BRAM，並保留 clock/reset/enable control sets。此階段完成不代表時序已通過；synth timing 使用估算互連，實際 net delay 必須等 placement 與 routing 才能確認。

本版板級 routed design 使用 12,411 LUT、9,029 FF、10 個 RAMB18、0 DSP。綜合後的 critical-path family 會依 endpoint primitive、pin type、fanin cone、logic delay 與 net delay 分類；若主要問題是邏輯深度，就調整 pipeline 或 Boolean structure；若主要問題是 control fanout / routing delay，就改善控制訊號 locality、FIFO clear 行為或 physical optimization。

2.2.4 Implementation、timing closure 與 signoff

Implementation 將 synthesized netlist 經過 optimization、placement、physical optimization 與 routing。保留流程的主要指令為：

Listing 2.1: 175 MHz implementation 主流程

```
opt_design      -directive ExploreWithRemap
place_design    -directive ExtraNetDelay_high
phys_opt_design -directive AggressiveExplore
route_design    -directive Explore
phys_opt_design -directive AggressiveExplore
```

判定不只看「implementation completed」。Setup 必須同時滿足 $WNS \geq 0$ 、 $TNS = 0$ 、 $setup\ FEP = 0$ ；hold 必須滿足 $WHS \geq 0$ 、 $THS = 0$ 、 $hold\ FEP = 0$ 。此外還要確認所有 routable nets 完成 routing、route error 為 0、DRC 與 methodology 無 Error/Critical Warning、unsafe CDC 為 0，且 `check\_timing critical categories` 為 0。

最終 routed 結果為 $WNS +0.029\ ns$ 、 $TNS\ 0$ 、 $setup\ FEP\ 0$ 、 $WHS +0.044\ ns$ 、 $THS\ 0$ 、 $hold\ FEP\ 0$ 、17,758/17,758 routable nets fully routed。這表示 175 MHz 的 implementation 與靜態 timing gate 已通過；它仍必須與後續功能與實板結果一起解讀，不能單獨證明 AES-GCM 數學結果正確。

2.3 Bitstream、實板驗證與 release evidence

Signoff routed DCP 通過後才產生 bitstream。燒錄前以 SHA-256 與 manifest 核對 bitstream identity，避免「報告分析 A、板子卻燒 B」。板級驗證再經 UART wrapper 將 NIST RSP case 送入 DUT，checker 逐筆比對 status、ciphertext/plaintext、tag 與預期 authentication failure；目前六模式 smoke 為 6/6，NIST 為 47,250/47,250，fail 0，預期 authentication failure 11,908 筆也全部得到正確分類。

最終 release 必須一起保留 RTL、constraints、SVA/UVM testbench、Tcl flow、synth/placed/routed DCP、timing/DRC/CDC reports、bitstream、board logs 與 hashes。Git commit 負責固定來源版本，artifact manifest 負責把來源、工具版本與輸出 identity 連接起來。如此才能回答「哪一版 RTL、用哪個 Vivado build、哪些 directives、產生哪個 bitstream、在板上跑了哪些 vectors」。

2.4 FPGA flow 與 ASIC flow 的界線

本報告雖以「IC 設計流程」描述數位設計方法，但實際 target 是 Xilinx Artix-7 FPGA。FPGA 的 LUT、FF、BRAM、clock network、placement 與 routing resources 已由晶片廠預先製造，Vivado 完成的是對既有資源的 mapping、place、route 與 bitstream generation；因此本專案沒有 standard-cell library characterization、DFT scan insertion、clock-tree synthesis、power-grid design、IR-drop/EM、LVS、GDSII 或 tape-out。

若未來將核心移植為 ASIC，前端的規格、演算法、RTL、UVM、SVA 與大部分 functional verification 可以沿用；後端則要把 FPGA implementation 替換成 logic synthesis、DFT、floorplan/power planning、standard-cell placement、CTS、routing、parasitic extraction、multi-corner STA、IR-drop/EM、DRC/LVS 與 GDS signoff。這個界線也說明：本報告的「implementation 通過」是 **FPGA implementation signoff**，不是 ASIC tape-out signoff。

第三章 AES 與 GCM 演算法原理

3.1 AES state、key size 與 round 數

AES 將 128-bit block 視為 4×4 byte state。初始先做一次 AddRoundKey；中間 round 依序做 SubBytes、ShiftRows、MixColumns、AddRoundKey；final round 省略 MixColumns。Key size 與 round 數如下：

表 3.1: AES key size 與 round 數

Key size	N_k (32-bit words)	N_r (rounds)
128 bit	4	10
192 bit	6	12
256 bit	8	14

3.2 SubBytes：有限體反元素與 affine transform

SubBytes 是 AES 唯一非線性 round transformation。數學定義為：先在 $GF(2^8)$ 中求 byte 的乘法反元素 (0 另行定義)，再做 affine transformation；AES 使用不可約多項式

$$m(x) = x^8 + x^4 + x^3 + x + 1. \quad (3.1)$$

目前 RTL 的 `sbox.v` 不是在每次運算時計算反元素，而是以完整 256-entry case table 表示 S-box。這是演算法上等價的 lookup-table 方法。實作中：

- shared `aes_block_engine` 有 16 個 registered S-box，Vivado 將其映射成 8 個 RAMB18；一個 clock 同時處理 state 的 16 bytes。
- 兩個 dedicated `aes_first_block_engine` 使用 16-way parallel combinational S-box，以 LUT 換取一 round/clock 的短 latency。
- key expansion 使用 4 個 registered S-box，映射成另外 2 個 RAMB18。

因此板級總計為 10 個 RAMB18。這個選擇保留了 BRAM-based S-box 的規則延遲，也讓 shared engine 的 S-box output 成為顯式 pipeline stage。Reference 資料夾中的 composite-field S-box 文獻說明另一種 area/speed 取舍，但目前 RTL 並未採用 composite-field inversion，不能把該方法誤寫為已實作。

3.3 ShiftRows : 固定 byte permutation

ShiftRows 將 state 第 0、1、2、3 列分別循環左移 0、1、2、3 bytes。此操作沒有乘法器或查表，只是 128-bit bus 的固定重新接線。RTL 的 `shift\_rows` function 直接重排 bit slices，因此理想上不消耗獨立 arithmetic resources；實際 delay 主要來自 routing 與前後 register/LUT 的組合。

3.4 MixColumns : $GF(2^8)$ 固定矩陣乘法

每一個 4-byte column 乘以下列固定矩陣：

$$\begin{bmatrix} b'_0 \\ b'_1 \\ b'_2 \\ b'_3 \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} b_0 \\ b_1 \\ b_2 \\ b_3 \end{bmatrix}_{GF(2^8)} . \quad (3.2)$$

RTL 以 `xtime` 實作乘 2：

$$\text{xtime}(a) = (a \ll 1) \oplus \begin{cases} 0\mathbf{x}1\mathbf{B}, & a_7 = 1, \\ 0, & a_7 = 0. \end{cases} \quad (3.3)$$

乘 3 則為 $\text{xtime}(a) \oplus a$ 。四個 column 完全平行，只有 shift/XOR/LUT，不使用 DSP。Final AES round 直接選擇 ShiftRows output，繞過 MixColumns。

3.5 AddRoundKey : 128-bit XOR

AddRoundKey 為

$$S_{r+1} = T(S_r) \oplus K_r, \quad (3.4)$$

其中 T 是 SubBytes/ShiftRows/MixColumns 的組合。硬體上是 128 個 parallel XOR。這一層本身不複雜，但 1,920-bit round-key store 的選擇 mux、round-index fanout 與 routing locality 曾是 timing 問題。現在 shared engine 保留四份 4-bit round selector，分別靠近每個 32-bit quarter，避免單一 selector 對 128 個 key bits 的高 fanout。

3.6 Key Expansion : RotWord、SubWord、Rcon

`aes_key_context.v` 先擴展並保存最多 15 組 128-bit round keys，共 1,920 bits。每個新 32-bit word 一般為

$$w_i = w_{i-N_k} \oplus w_{i-1}, \quad (3.5)$$

在 $i \bmod N_k = 0$ 時，對上一個 word 做 RotWord、SubWord 與 Rcon；AES-256 在 $i \bmod 8 = 4$ 額外做 SubWord。Rcon 以同一個 $GF(2^8)$ `xtime` 更新。

Key expansion 是 iterative FSM，normal word 約 2 clocks，S-box word 約 4 clocks。由 RTL state transition 推得 commit-to-ready latency 為 AES-128 約 100 clocks、AES-192 約 108 clocks、

AES-256 約 130 clocks ; key ready 後 , round keys 由後續所有 records 重用。這能把 key-schedule work 從 steady-state data path 移除 , 代價是 1,920-bit storage 與前置 key-load latency 。

3.7 GCM : CTR confidentiality 與 GHASH authentication

GCM 將 AES counter mode 與 universal hash 結合。先計算

$$H = E_K(0^{128}), \quad (3.6)$$

$$J_0 = \begin{cases} IV \parallel 0^{31} \parallel 1, & |IV| = 96, \\ \text{GHASH}_H(IV), & \text{otherwise,} \end{cases} \quad (3.7)$$

$$C_i = P_i \oplus E_K(\text{inc32}(J_0, i)), \quad (3.8)$$

$$T = \text{MSB}_t(\text{GHASH}_H(A, C) \oplus E_K(J_0)). \quad (3.9)$$

CTR 只需要 AES encryption datapath , 解密同樣以 ciphertext XOR keystream 還原 plaintext。GHASH 的 recurrence 為

$$Y_i = (Y_{i-1} \oplus X_i) \cdot H \quad \text{in } GF(2^{128}), \quad (3.10)$$

field polynomial 為

$$x^{128} + x^7 + x^2 + x + 1. \quad (3.11)$$

3.8 目前 GHASH 的 16-bit digit-serial 架構

`ghash16.v` 每次處理 X 的 16 bits , 完成 128-bit multiplication 需要 8 個 digit clocks。Start/load 是獨立 clock , 因此外部 initiation interval 為 9 clocks。內部不是串接 16 個 runtime shifts , 而是同時建立 $T^0(V)$ 到 $T^{16}(V)$ 的 closed-form linear transforms ; 依 $X[127:112]$ 選擇 16 個 field terms , 再用 balanced XOR tree 做 $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ reduction。這將 XOR depth 壓到四層 , 避免長 XOR chain。

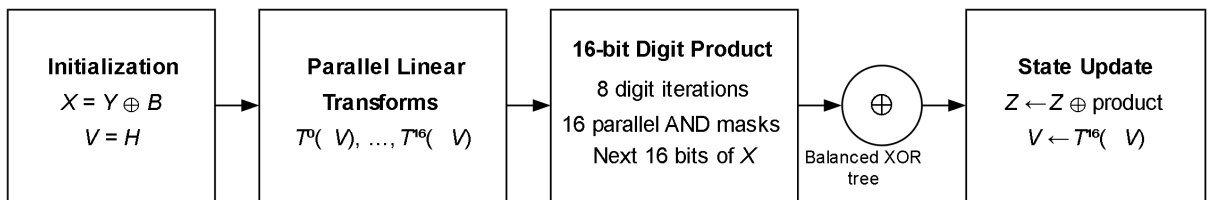


图 3.1: 目前 RTL 的 16-bit digit-serial GHASH 資料路徑

Reference 文獻中的 Karatsuba-Ofman multiplier、四路 parallel GHASH 與深 pipeline 可達更高吞吐率 , 但會增加 area、pipeline state 與 verification complexity。目前設計沒有使用 Karatsuba ; 在 175 MHz 下 , GHASH raw block capacity 約為

$$\frac{128 \times 175 \text{ MHz}}{9} = 2.489 \text{ Gbit/s}, \quad (3.12)$$

已高於 8-bit stream 的理論上限 1.4 Gbit/s , 因此現階段不需要以更大 multiplier 換 area。

第四章 RTL 硬體架構與平行化

4.1 整體資料流

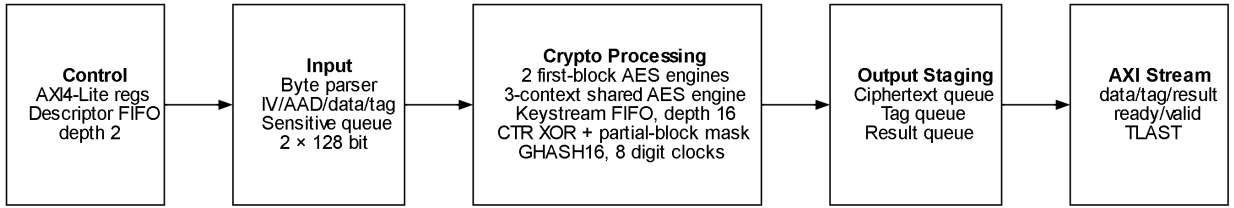


图 4.1: AES-GCM core 的主要資料流、queues 與 crypto engines

4.2 AES engines : 三 context pipeline 加兩個 first-block engines

主 `aes_block_engine` 以 16 個 registered S-box 共用一條 round datapath，保存三份 state contexts。每個 context 的 round recurrence 為三個 pipeline clocks：

1. issue context state 與 round index；
2. registered S-box output，同時選擇 round key；
3. ShiftRows / MixColumns / AddRoundKey 回寫 context。

三個 contexts 交錯執行，故 single-request latency 約為 $3N_r$ clocks，但 steady-state completion interval 降為 N_r clocks：AES-128/192/256 分別為 10/12/14 clocks。ready 具有 finish-now look-ahead，可在 context 完成的同一 edge 接受替代 request。

R55 為了解決 175 MHz 下最前面兩個 CTR blocks 的 first-result latency，新增兩個 `aes_first_block_engine`，分別專門處理 block index 0 與 1。每個 engine 一 round/clock，latency 為 10/12/14 clocks。後續 blocks 仍由 shared engine 處理；這不是複製三條完整 AES-GCM lanes，而是針對 startup bubble 的局部平行化。

表 4.1: 核心 crypto unit 的 clock latency 與 initiation interval

Unit	AES-128	AES-192	AES-256	說明
Dedicated first-block engine latency	10	12	14	一 round/clock , 兩個 engines 處理 index 0/1
Shared engine nominal single-block latency	30	36	42	三-stage round recurrence , 無其他 context 時
Shared engine steady-state block II	10	12	14	三 contexts interleaving
GHASH latency	8 digit clocks			外部 request II = 9 clocks
8-bit stream block transfer	16 clocks / 128-bit block			理論 raw width limit = 1.4 Gbit/s

4.3 兩筆 sensitive data queue 與 throughput 改善

第一個 AES keystream result 的 latency 長於 byte stream 完成第一個 128-bit block 的時間。舊架構會在 block boundary 等待 keystream , 造成每筆 record 的 bubble。R55 加入兩筆 128-bit sensitive data queue : input blocks 先排隊 , 等 ordered keystream FIFO head 到達再 XOR、GHASH 或寫 plaintext bank。Queue 支援同 cycle enqueue/dequeue , 並在 protocol abort 或 ZEROIZE 同步清除 plaintext/ciphertext-sensitive payload。

這個改動的重點不是提高 clock frequency , 而是提高**每個 clock 的有效工作比例** : 把 first-result latency 與 byte admission overlap。搭配兩個 first-block engines , 100 筆 1024-bit records 的 cycles 下降到 encrypt 16,820、decrypt 17,039 , 三種 key sizes 都超過 1 Gbit/s。

4.4 FIFO 與 backpressure

重要 queue 深度如下 : descriptor 2、AES request 4、AES metadata 4、AES result 4、keystream 16、GHASH request 4、ciphertext 4、tag packets 2、authenticated plaintext release 2。所有接口使用 valid/ready ; AES engine 本身沒有 output ready , 故 result FIFO 吸收完成脈衝。Decrypt plaintext 先存於兩個 packet banks , tag comparison 成功後才進 release queue , 避免 unauthenticated plaintext leakage。

4.5 ZEROIZE 與 security locality

過往 timing family analysis 顯示，ZEROIZE/control nets 對 FIFO CE/WE、prep registers 與 round-key controls 的高 fanout/locality 是主要 physical risk。最終 RTL 採用 local registered control、one-hot round-key word write enables、分散的 round selector、同步 reset/zeroize priority 與 queue-local clear。Round48/49/53 security regressions、fast-zeroize、data-buffer clearing 與 NIST simulation suites 均通過。

第五章 Clock、cycle 與吞吐率

5.1 175 MHz 時脈產生

Arty A7 板上輸入為 100 MHz。MMCME2_BASE 設定 CLKFBOUT_MULT_F=7、CLKOUT0_DIVIDE_F=4，產生 175 MHz core clock：

$$f_{core} = 100 \text{ MHz} \times \frac{7}{4} = 175 \text{ MHz}, \quad T_{clk} = \frac{1}{175 \text{ MHz}} = 5.714 \text{ ns}. \quad (5.1)$$

Global clock 使用 1 個 MMCM 與 2 個 BUFG；core clock 有 9,324 個 clock-pin loads，分布於 X0Y2、X1Y2、X0Y3 與 X1Y3 四個 active clock regions。

5.2 吞吐率公式與測試定義

吞吐率必須用 payload bits、測量 clocks 與 clock frequency 同時計算：

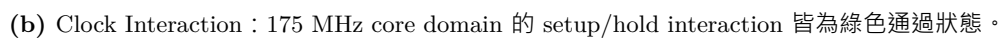
$$\text{Throughput} = \frac{N_{payload \text{ bits}} \cdot f_{clk}}{N_{cycles}}. \quad (5.2)$$

Testbench 每個 mode 執行 100 筆 records，每筆 1024-bit data，故加密為

$$\frac{100 \times 1024 \times 175 \times 10^6}{16820} = 1.065398 \times 10^9 \text{ bit/s}. \quad (5.3)$$

解密為

$$\frac{100 \times 1024 \times 175 \times 10^6}{17039} = 1.051705 \times 10^9 \text{ bit/s}. \quad (5.4)$$



15

175 MHz RTL Throughput					
100 × 1024-bit records; acceptance target > 1.000000 Gbit/s					
Key mode	Operation	Cycles	Throughput	Margin vs. target	Result
AES-128	Encrypt	16,820	1.065398 Gbit/s	+6.54%	PASS
AES-128	Decrypt	17,039	1.051705 Gbit/s	+5.17%	PASS
AES-192	Encrypt	16,820	1.065398 Gbit/s	+6.54%	PASS
AES-192	Decrypt	17,039	1.051705 Gbit/s	+5.17%	PASS
AES-256	Encrypt	16,820	1.065398 Gbit/s	+6.54%	PASS
AES-256	Decrypt	17,039	1.051705 Gbit/s	+5.17%	PASS
Boundary: RTL simulation cycle measurement, not UART or board-level throughput.					

图 5.2: 175 MHz 下的六模式 RTL throughput ; 三種 key size 皆大於 1 Gbit/s

Key size 不再改變這個 packet-level measured throughput，是因為最前兩個 blocks 被 dedicated engines 隱藏，後續 shared engine 的 14-clock worst-case II 仍快於 16 clocks/128-bit 的 byte-stream arrival rate。GCM/FIFO/record overhead 使實測值低於 8-bit stream 理論上限 $8 \times 175 = 1.4$ Gbit/s。

5.3 Latency 與 throughput 的差異

Latency 是一筆工作從 accept 到 result 的 clocks；throughput 是 steady state 每秒完成多少 payload。Pipeline 可以增加單筆 latency，卻同時降低 initiation interval。例如 shared AES 單筆 nominal latency 可為 30/36/42 clocks，但三 context interleaving 後平均每 10/12/14 clocks 完成一個 block。兩個 first-block engines 則專門縮短 record startup latency。

5.4 實板 UART throughput

板上 NIST wrapper 的 UART 是 115200 baud、8N1。每 8-bit byte 需 10 line bits，因此純 UART payload ceiling 為

$$115200 \times \frac{8}{10} = 92.16 \text{ kbit/s}. \quad (5.5)$$

每個 frame 還有 command/type 與 16-bit bit-length 共 3 bytes header；例如 128-byte payload 單一 frame 的 wire efficiency 為

$$\frac{128}{128 + 3} \times 80\% = 78.17\%. \quad (5.6)$$

完整 hardware NIST run 共執行 1,208.505 s，平均 39.098 cases/s。這是包含向量長度、frame 數量、host Python、serial turnaround 與 status/data/tag response 的 end-to-end 測試速率，不代表

crypto datapath 的上限。若要在板上直接量測 1 Gbit/s，應改用高速 AXI DMA、Ethernet、PCIe 或純硬體 on-chip counter，不應透過 UART。

第六章 Synthesis、資源使用率與功率

6.1 Synthesis 的目的

Synthesis 將 Verilog RTL elaboration 成 technology-mapped netlist，推導 FF、LUT、LUTRAM、BRAM、CARRY、MMCM 等 primitives，並進行 Boolean optimization、memory inference 與初步 timing estimate。它回答「邏輯能否被 FPGA 資源實現、會用多少資源、粗估 critical path」，但**不能證明實體 timing closure**；真正的 wire delay、placement congestion 與 hold repair 必須在 place/route 後判定。

6.2 板級 routed 資源

表 6.1: Arty A7-100T routed resource utilization

Resource	Used	Available	Utilization	說明
Total LUT	12,411	63,400	19.58%	11,333 logic + 1,078 LUTRAM
FF	9,029	126,800	7.12%	包含 core、AXI、UART、reset/heartbeat
BRAM36 equivalent	5	135	3.70%	實體為 10 個 RAMB18
DSP	0	240	0.00%	AES/GHASH 皆用 XOR/LUT/BRAM
MMCM	1	6	16.67%	100 MHz 轉 175 MHz
BUFG	2	32	6.25%	core clock 與 MMCM feedback

Arty A7-100T Final Routed Resource Utilization				
Resource	Used	Device capacity	Utilization	Relative scale (0-25%)
LUT	12,411	63,400	19.58%	
FF	9,029	126,800	7.12%	
BRAM36 equivalent	5	135	3.70%	
DSP	0	240	0.00%	
Used values: final routed DCP. Capacities: XC7A100T device specification.				

图 6.1: LUT、FF、BRAM 與 DSP 使用率圖；數據來自 final routed DCP

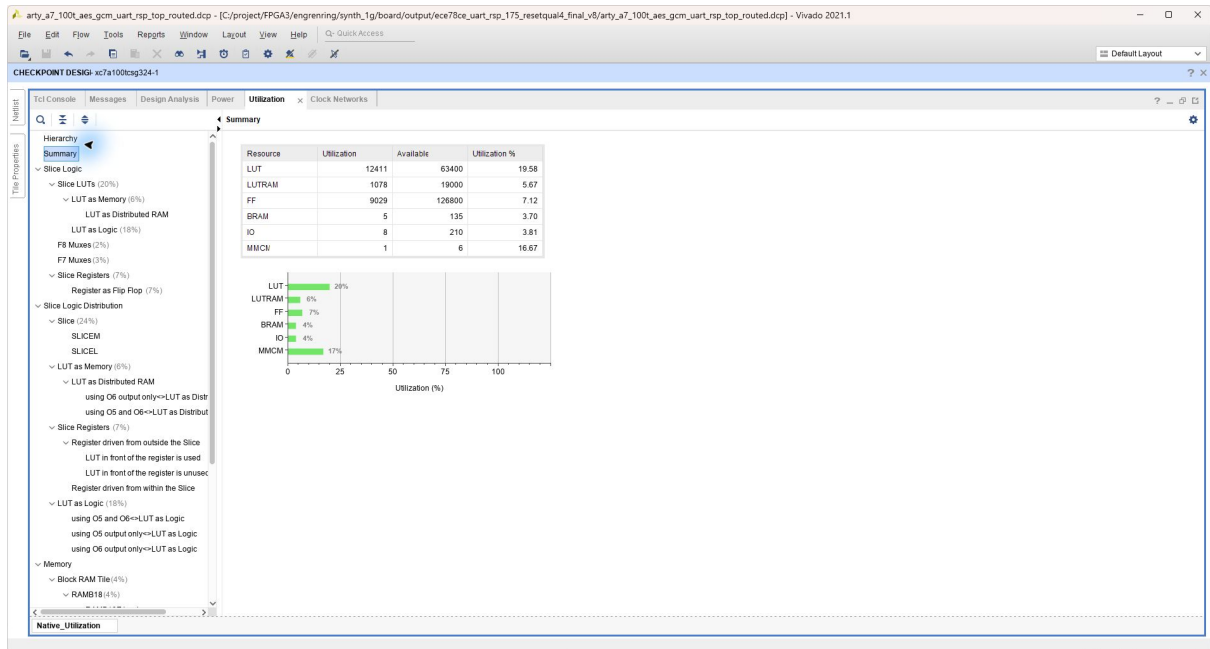
6.3 主要 hierarchy 資源分布

表 6.2: Final routed utilization by hierarchy

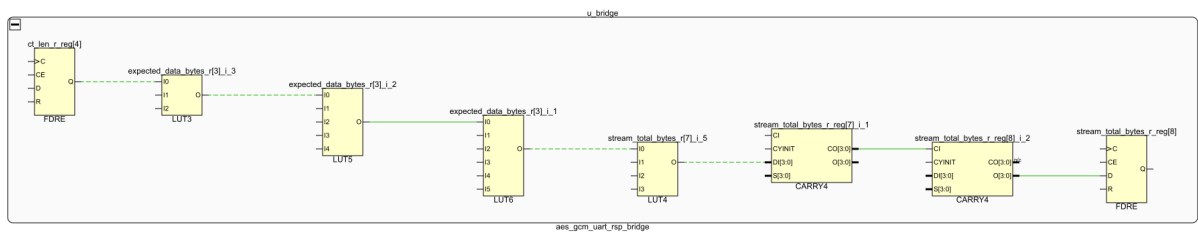
Hierarchy	LUT	LUTRAM	FF	RAMB18
Board top total	12,411	1,078	9,029	10
u_aes_gcm	11,323	964	8,059	10
u_aes_gcm/u_core	11,192	948	7,670	10
u_core/u_key_context	2,930	0	2,344	2
u_core/u_ghash	1,442	0	648	0
u_core/u_aes_engine	671	0	719	8
u_core/u_aes_first_engine	781	0	150	0
u_core/u_aes_second_engine	783	0	150	0
u_bridge	1,086	114	694	0

6.4 使用率降低：UART wide-register arrays 改成 byte memories

早期 board v3 將最大 1024-bit input/capture payload 實作為大量 FF，並用 dynamic index 寫入。這同時造成 wide mux、reset/control fanout、LUT 與 FF 暴增。v4 將 input、data-capture、tag-capture 改為 8-bit distributed byte memories，保留相同 UART protocol。結果如下：



(a) Vivado 原生 Utilization Summary 與使用率長條圖。



(b) Synthesis checkpoint 的 technology-mapped worst setup path schematic ; 此時 interconnect 尚為估計值。

图 6.2: Synthesis 後的原生資源使用率與 critical-path netlist 視圖。

表 6.3: UART bridge memory refactor 前後資源與 synth timing

版本	LUT	FF	RAMB18	Synth WNS	Synth FEP
v3 wide FF arrays	22,436	15,430	12	−1.192 ns	1,489
v4 byte RAM	12,388	8,996	10	−0.298 ns	292
Final v8c strict audit	12,411	9,029	10	+0.029 ns	0

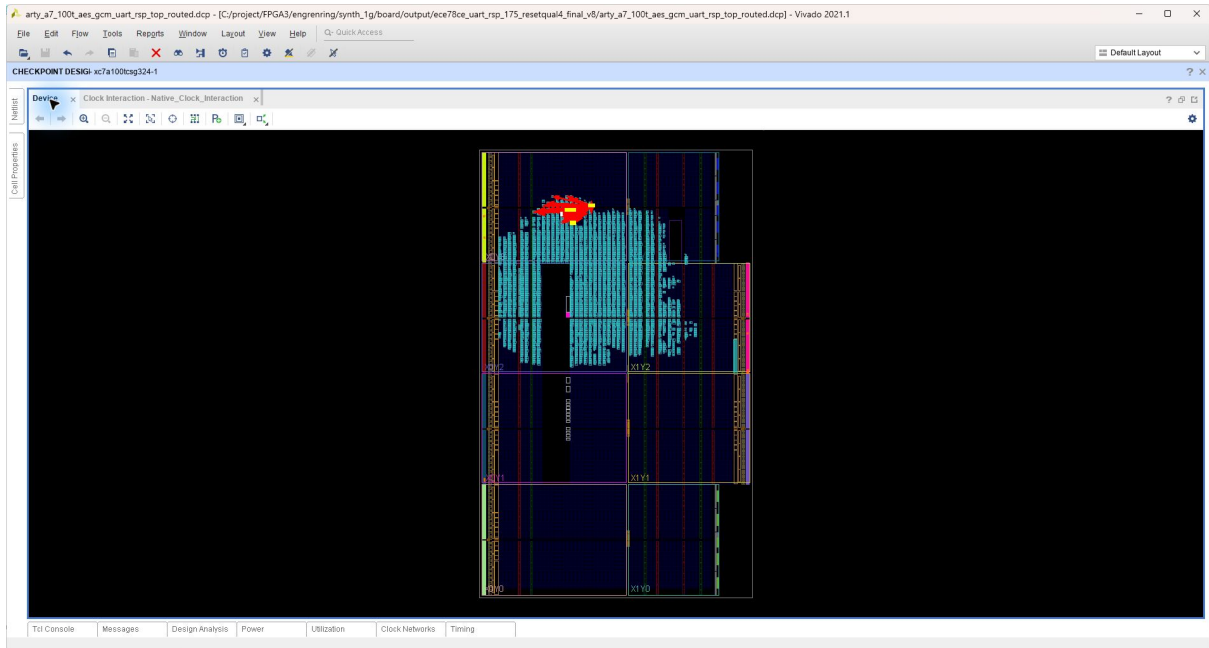
相對 v3 · v4 synth 約減少 10,048 LUT (44.8%) 與 6,434 FF (41.7%) 。這是明確的 utilization reduction 。它也大幅降低 timing violations ；但由於 v3 沒有可比較的 final routed power report ，本報告不宣稱精確的功率下降百分比。

6.5 Placement 分布與 routing 可視化

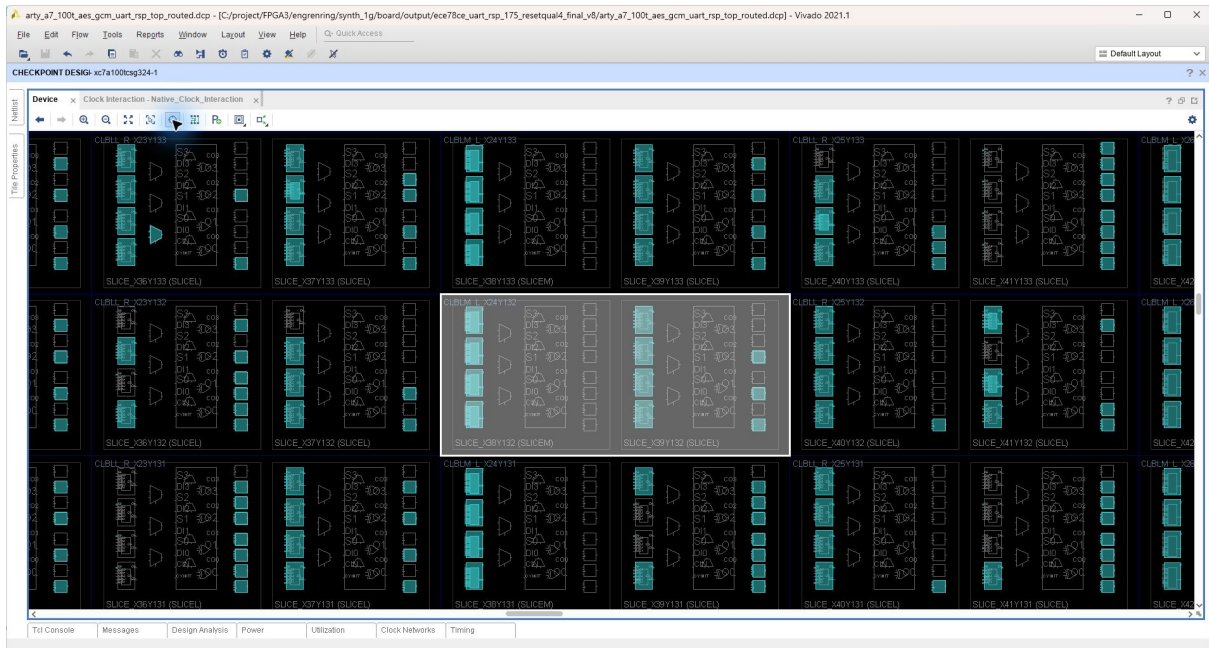
Active Clock-Region Placement Summary	
X0Y3 FF 2,881 LUTM 660 RAMB 0 MMCM 0	X1Y3 FF 676 LUTM 276 RAMB 0 MMCM 0
X0Y2 FF 4,071 LUTM 892 RAMB 4 MMCM 0	X1Y2 FF 1,401 LUTM 438 RAMB 0 MMCM 1
FF denotes clock loads; LUTM/RAMB/MMCM are from the routed clock-utilization report. This is not a Device routing screenshot.	

图 6.3: Final routed DCP 的 active clock-region placement 摘要。圖中 FF 為該 clock 的實際 clock loads ；LUTM/RAMB 欄位取自 Vivado clock-utilization report ，並非全器件 primitive 總數。

完整 placement 與逐條 routing wire 可在 Vivado 開啟 final routed DCP 後於 Device view 檢查。本報告提供可列印的 clock-region 圖；interactive DCP 才是最完整的「落線圖」來源。Final audit 的 route status 為 17,758 個 routable nets 全部 fully routed 、0 routing error 。



(a) 完整 Device placement ; 青色為已放置 cells · 紅 / 黃色為 highlighted setup path 。



(b) Critical region 的 tile-level placement 細節 。

圖 6.4: Vivado 原生 implementation placement 視圖 。

6.6 功率估算

表 6.4: Vivado routed vectorless power estimate

項目	Power	項目	Power
Total on-chip	0.461 W	Dynamic	0.363 W
Device static	0.099 W	Junction temperature	27.1 °C
Signals	0.113 W	MMCM	0.090 W
Slice logic	0.078 W	Clocks	0.041 W
Block RAM	0.035 W	I/O	0.005 W

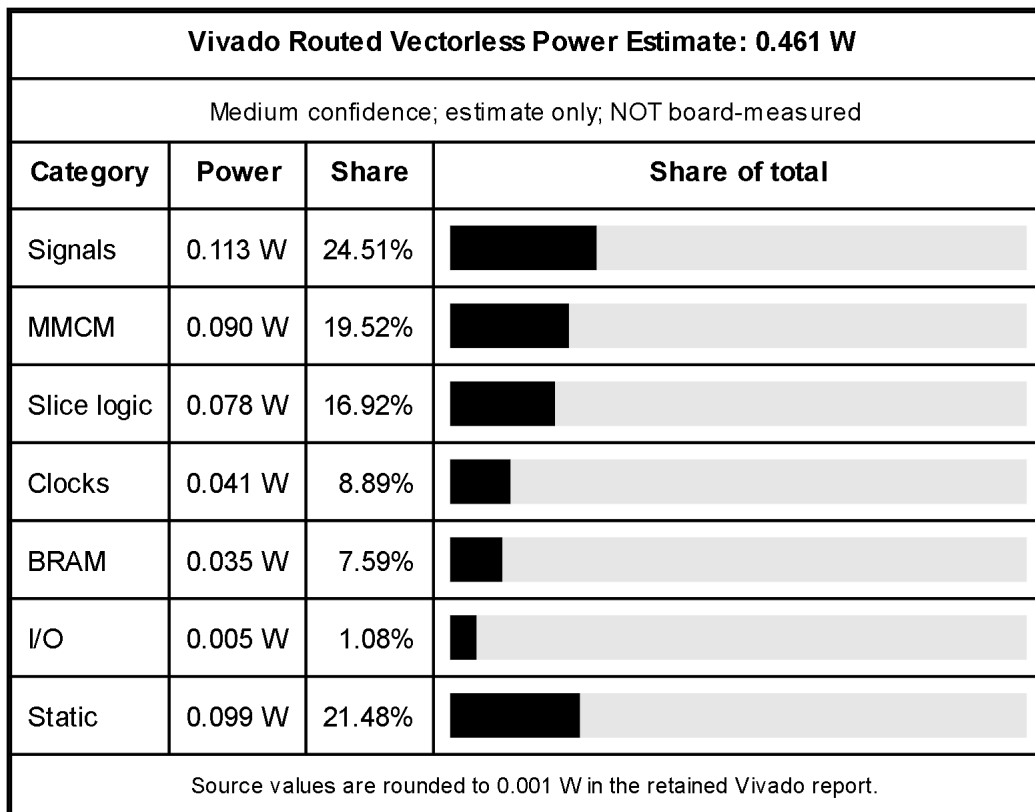


图 6.5: Final routed vectorless power breakdown

Power confidence 為 Medium : implementation state 與 clock activity 為 High confidence , 但 I/O 與 internal node activity 缺少實際 SAIF/VCD 。要得到較可信的 active crypto power , 應以代表性 AES-GCM simulation 產生 SAIF , 再執行 `read\_saif` 與 `report\_power` ; 若要板級實測 , 需量測 Arty 電源 rail 電流並扣除板上 regulator/USB/LED 等非 IP 功耗。

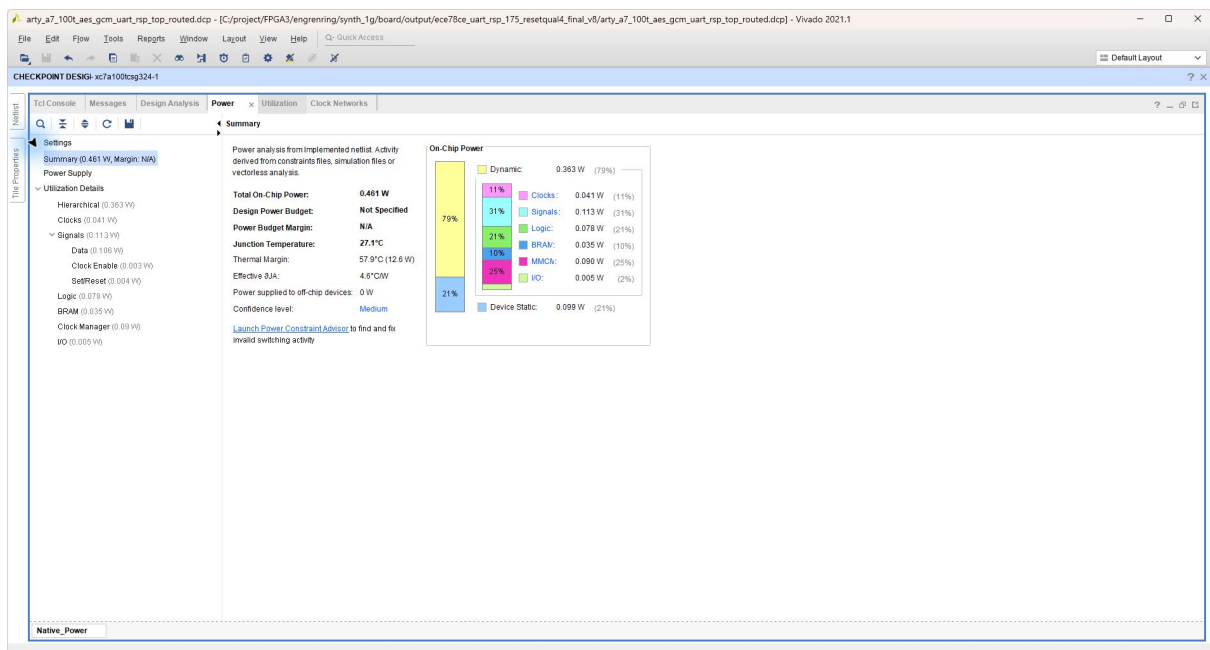


图 6.6: Vivado 原生 Power Summary : 總 on-chip power 為 0.461 W , 其中 dynamic 0.363 W 、 device static 0.099 W ; confidence level 為 Medium 。

第七章 Implementation 與 timing closure

7.1 Vivado flow 與 Tcl 的角色

Tcl 不是 AES 演算法的一部分，而是 Vivado 的可重現 flow driver。它精確控制 RTL/XDC 載入、synthesis、place、physical optimization、route、reports、DCP hash gate、DRC gate 與 bitstream。使用 Tcl 可以避免 GUI 手動點選造成 directive、constraint 或輸出路徑不一致。

板級 fresh flow 的主要階段為：

```
synth_design -top arty_a7_100t_aes_gcm_uart_rsp_top \  
    -part xc7a100tcsg324-1  
opt_design -directive ExploreWithRemap  
place_design -directive ExtraNetDelay_high  
phys_opt_design -directive AggressiveExplore # pre-route  
route_design -directive Explore  
phys_opt_design -directive AggressiveExplore # post-route  
report_timing_summary  
report_utilization -hierarchical  
report_route_status  
report_drc  
report_methodology  
report_cdc -details  
check_timing -verbose  
write_checkpoint  
write_bitstream
```

v8 是 fresh synthesis/place/route，沒有重用舊 placed/routed checkpoint。完成後，audit_routed_board 以預期 SHA-256 開啟 immutable routed DCP，在獨立的 resetqual4_audit_v8c 目錄重新產生 timing、pulse-width、route、DRC、methodology、CDC、check_timing、utilization、clock 與 power 證據，並輸出 bit/bin。Audit script SHA-256 為 62C653E5E49723A51726B92B0D475BA11C55F24A3648D593 輸入 routed DCP SHA-256 為 A91758ED87EB56648EEF0CAB67B2C50D198D00A2D5088E624B507687D99C016B。

中間的 v8b 結果因 audit gate 實作誤判而被拒絕；這是 gate logic 的 false rejection，不是 routed DCP 的 timing 或 legality fault。修正 gate 後，v8c 對同一個 hash-matched routed DCP 完成 strict audit 並通過所有 hard gates。

7.2 Timing iteration 結果

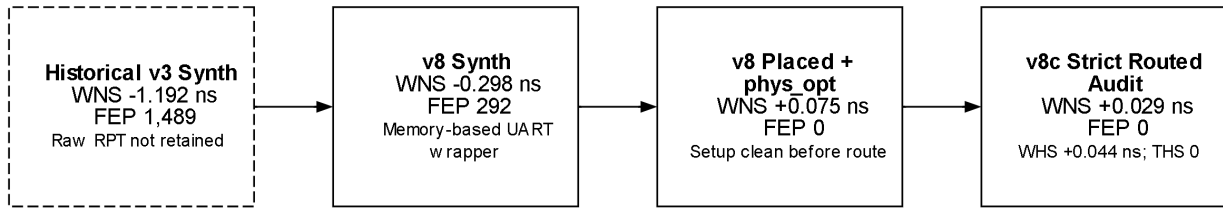


圖 7.1: Board wrapper 從 wide registers 到 final route 的 timing 收斂

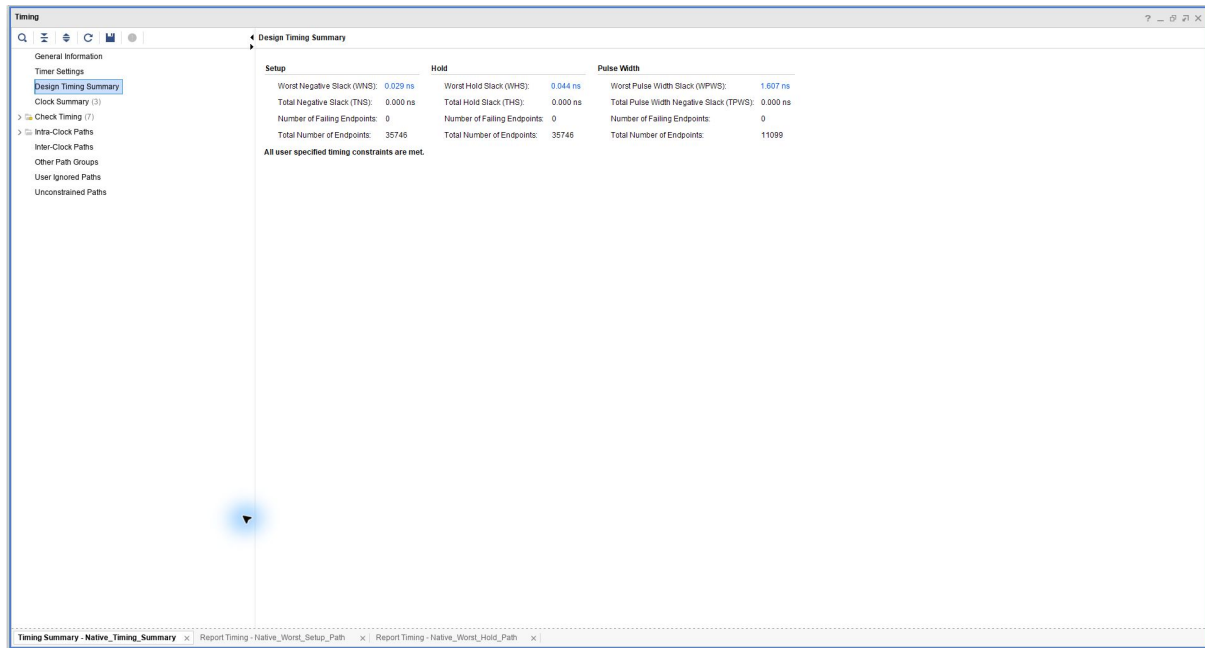
表 7.1: Timing iteration 詳細數據

Stage	WNS	TNS	Setup FEP	WHS	THS	Hold FEP
v3 synthesis	-1.192	-1506.136	1,489	+0.049	0	0
v8 synthesis	-0.298	-11.728	292	+0.045	0	0
v8 placed + phys_opt	+0.075	0	0	-0.082	-0.364	9
Final v8c strict audit	+0.029	0	0	+0.044	0	0

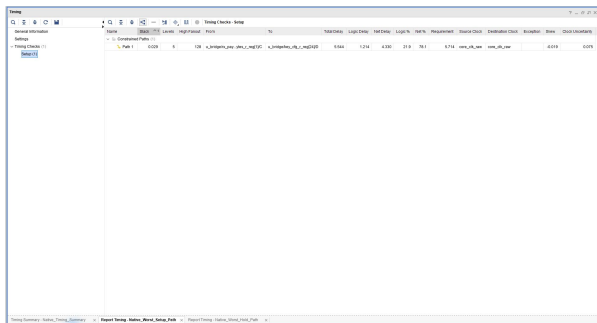
Synthesis 未通過並不表示最終一定失敗，因為其 interconnect estimate 尚未有真實 placement。Physical optimization 讓 setup 轉正，但 placement checkpoint 的 hold 仍有 9 個 failing endpoints；route 與 post-route physical optimization 加入實際 routing/hold repair 後，setup/hold 同時通過。Final strict audit 另確認 pulse-width WPWS +1.607 ns、TPWS 0、failing endpoints 0；timing report 明確顯示「All user specified timing constraints are met」。

7.3 175 MHz final scope 與 throughput 補償

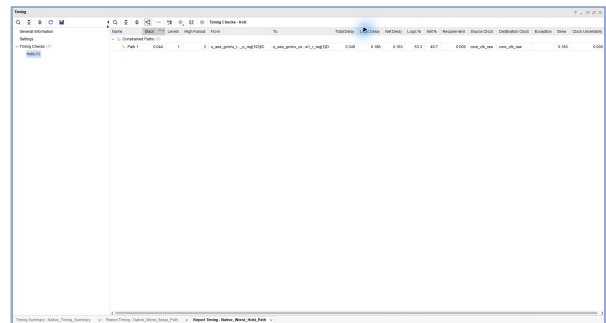
本版 production target 固定為 175 MHz。為避免 clock target 調整後 throughput 低於 1 Gbit/s，設計以兩個 first-block engines 與兩筆 data queue 消除 startup stalls，以提高每個 clock 的有效工作比例。Final v8c strict audit 與六模式 throughput regression 分別驗證 timing 與 > 1 Gbit/s 的 RTL payload throughput。



(a) Design Timing Summary : setup、hold 與 pulse width 均為綠色通過。



(b) Worst setup path · WNS +0.029 ns。



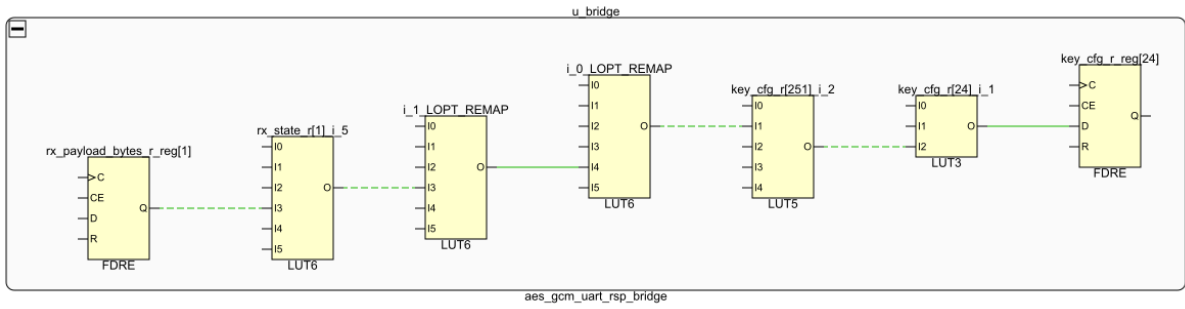
(c) Worst hold path · WHS +0.044 ns。

图 7.2: Vivado 原生 post-route timing signoff 視圖。

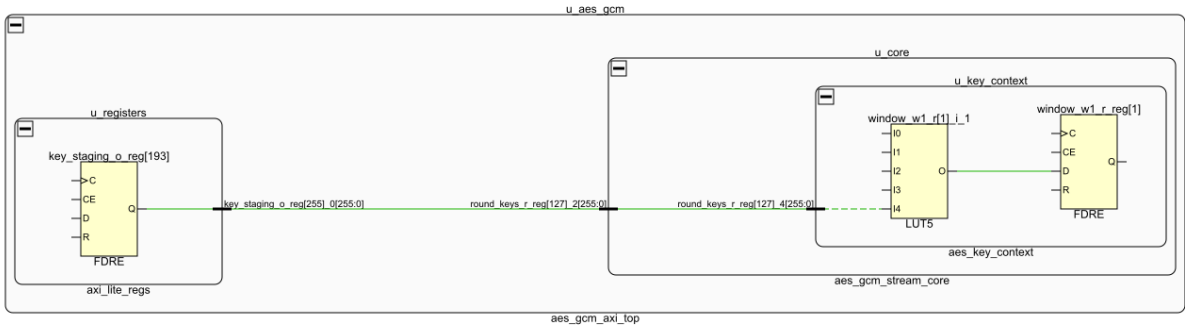
7.4 Final route 與 DRC

表 7.2: Route、DRC 與 methodology 摘要

檢查	結果	說明
Logical nets	25,065	7,307 不需 routing ; 17,758 routable
Fully routed nets	17,758	100%
Routing errors	0	無 failed/unrout- ed/overlap nets
DRC Error	0	Hard gate pass
DRC Critical Warning	0	Hard gate pass
Methodology Error / Critical Warning	0 / 0	2 個一般 SYNTH-6 Warning
CDC	2 safe / 0 unsafe	reset request 與 UART RX 皆為 depth-2 ASYNC_REG synchro- nizer
check_timing	10 critical categories = 0	async/status in- put/output false-path allowlists = 2 / 5
Pulse width	WPWS +1.607 ns / TPWS 0	failing endpoints 0

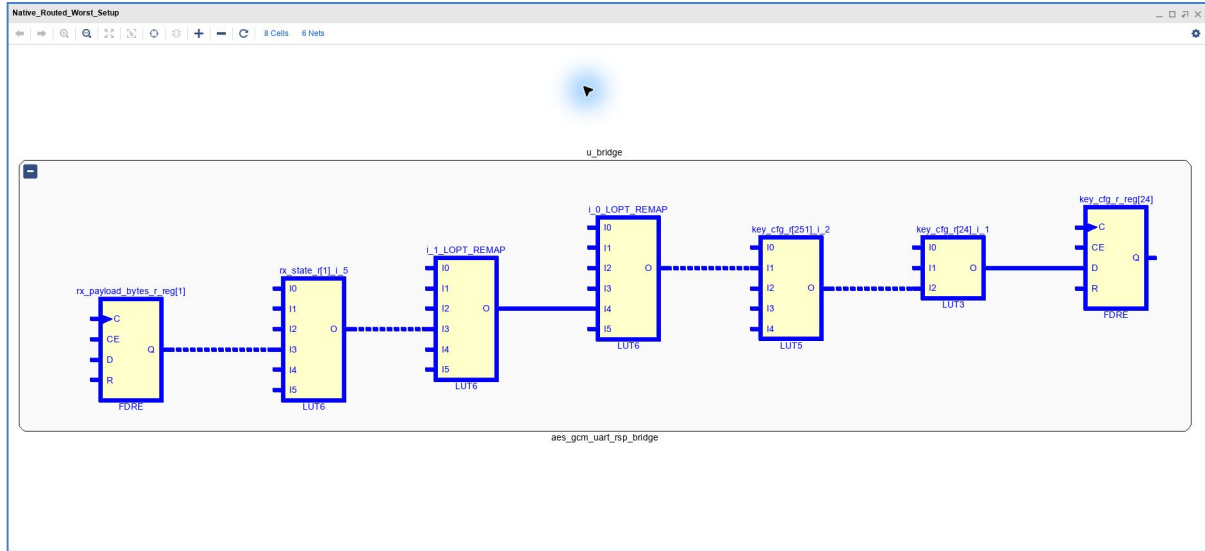


(a) Routed worst setup path : bridge payload register 經 LUT chain 到 key configuration register °

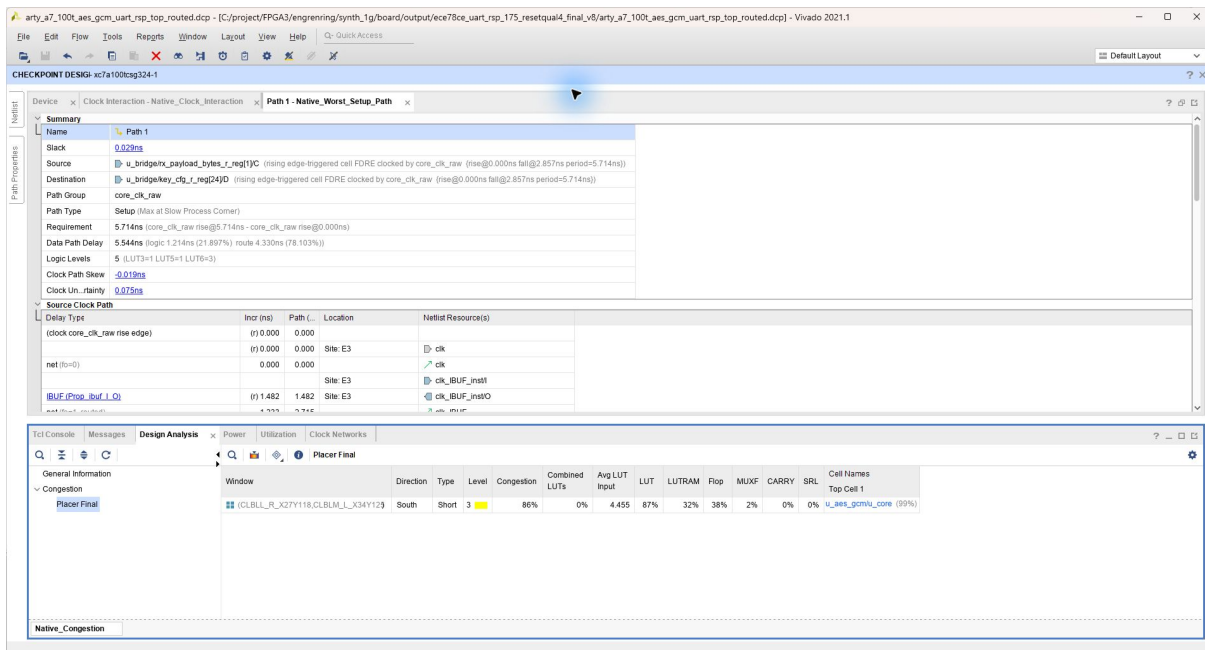


(b) Routed worst hold path : key staging register 到 key-context window register °

图 7.3: Vivado `write\_schematic` 匯出的 post-route setup / hold critical paths °

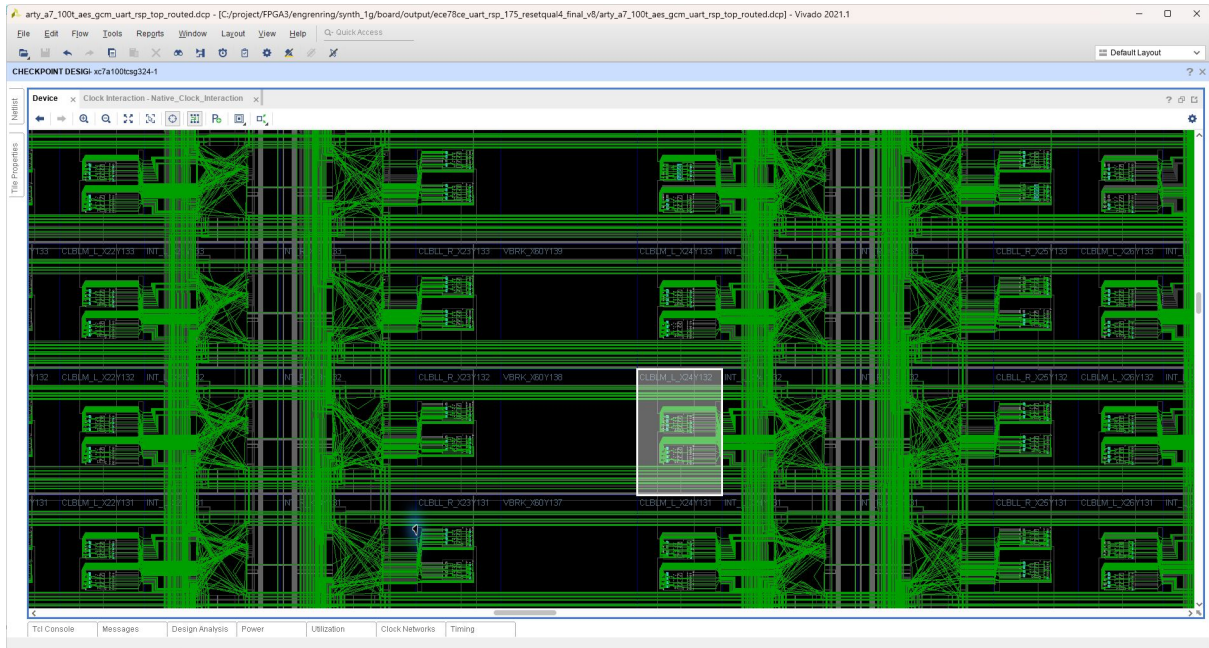


(a) Vivado GUI 中的 routed critical-path schematic。

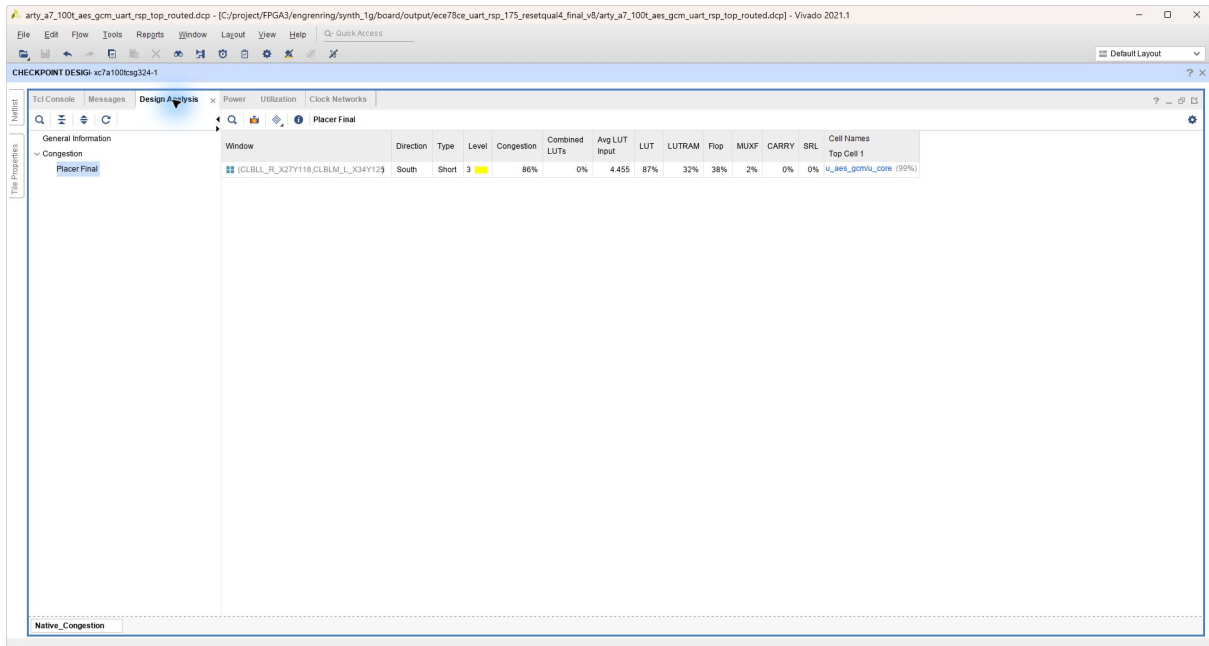


(b) Path 1 詳細 delay breakdown ; total data path 5.544 ns · 其中 net delay 4.330 ns · 占 78.1%。

圖 7.4: Worst setup path 的邏輯結構與實體 delay 證據。



(a) Device view 的 tile-level routing resources 與實際 wire segments。



(b) Design Analysis congestion view ; 報告中沒有高於 level 3 的 congestion window。

圖 7.5: Vivado 原生 routing resources 與 congestion analysis。

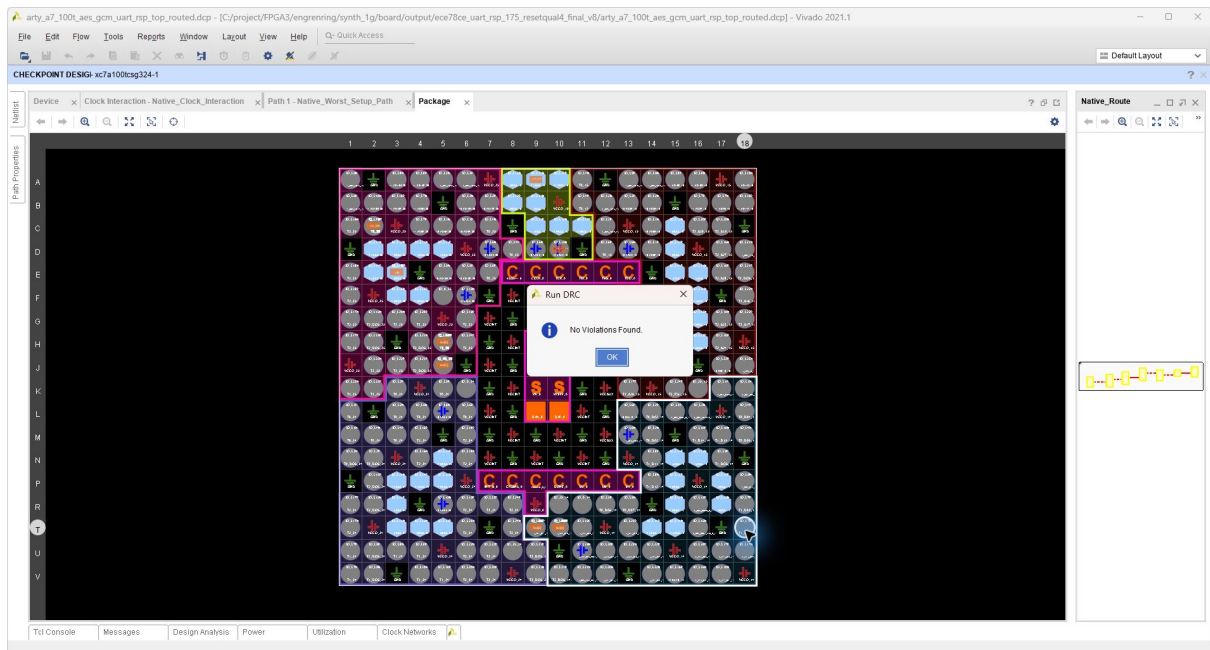


图 7.6: Vivado 原生 Run DRC 结果 : No Violations Found。

第八章 Timing 問題、同步化選擇與演算法 / 微架構改善

本章把前面分散在演算法、RTL 架構、吞吐率與 implementation 章節的重點集中在同一處，直接回答三個問題：遇到 Timing violation 時採用同步還是非同步設計、實際如何把 setup/hold 收斂，以及哪些演算法等價實作與硬體排程改善了吞吐率。核心原則是：**AES-GCM 的數學定義不變**，175 MHz crypto datapath 採同步設計，非同步只存在於受控的板級輸入邊界。

8.1 同步還是非同步：本設計的實際選擇

AES、GHASH、key expansion、FIFO、AXI4-Lite、AXI4-Stream 與 UART bridge 都由同一個 175 MHz `core_clk` 正緣驅動。組合 S-box、XOR 與有限體運算位於同步 register boundaries 之間，因此仍然必須在 5.714 ns clock period 內滿足 STA；它們不是 clockless asynchronous datapath。

表 8.1: 同步 / 非同步邊界與處理方式

訊號或功能	採用方式	原因與 Timing 意義
AES、GHASH、key expansion、FIFO、AXI 與 ZEROIZE	單一 175 MHz synchronous domain	所有 state、queue count、round state 與 scrub 動作都在 <code>posedge core_clk</code> 更新，可由 setup/hold STA、SVA 與 cycle-accurate simulation 檢查。
<code>rst_btn</code> 與 MMCM LOCKED	raw run request 先進 2-stage <code>ASYNC_REG</code> synchronizer	第一級可能發生 metastability；第二級在送入功能控制前提供一個 clock period 的收斂時間。這是同步器，不是非同步 AES 演算法。
Distributed <code>aresetn</code>	synchronizer 後接 4-stage consecutive-high qualifier	assertion 與 release 都只在 <code>core_clk</code> 正緣發生；避免短 pulse/chatter 誤解除 reset，也避免 S-box BRAM 取得 asynchronous reset control。
UART RX pin	2-stage <code>rx_meta</code> → <code>rx_sync</code> synchronizer	非同步 serial pin 先同步化；後續 start/data/stop sampling FSM 全部使用 <code>rx_sync</code> 。
STA exceptions	只切 raw asynchronous port path	XDC 只對 <code>rst_btn</code> 、 <code>uart_txd_in</code> 到第一級 FF 的 top-level path 使用 false path；同步器後續 FF-to-FF 與全部內部 crypto paths 仍受 STA。

Reset 的實際順序為

$$\text{rst_btn} \wedge \text{MMCM LOCKED} \longrightarrow 2\text{-FF synchronizer} \longrightarrow 4\text{-stage qualifier} \longrightarrow \text{synchronous aresetn.} \quad (8.1)$$

Directed reset simulation 證明 1–3 個 sampled clocks 的 pulse 與 alternating chatter 都不會誤解除 reset；連續有效 request 在第 6 個 sampled edge release，無效 request 在第 3 個 sampled edge assertion。Routed CDC audit 只找到預期的 reset request 與 UART RX 兩條 depth-2 safe synchronizers，unsafe CDC category 為 0。Qualifier 最後一級因 distribution fanout 被 implementation physical replication 成 4 個 FF，但它們仍來自同一個 logical reset source，不是四個獨立 reset 或 clock domains。

把內部 AES/GHASH critical path 宣告成 false path，只會隱藏 violation，不會縮短真實 propagation delay；改成 clockless asynchronous handshake 也不會自動消除 LUT depth、fanout 或 routing delay，反而會增加 CDC、metastability、驗證與 BRAM inference 風險。因此本設計以同步 pipeline、register locality 與可檢查的 valid/ready protocol 完成 closure。

8.2 Timing 問題、根因與收斂過程

最早的 board v3 把最大 1024-bit UART input/capture payload 實作為大量 FF，並以 dynamic index 寫入。這形成 wide mux、large reset/control cone 與高 fanout。後續版本雖已大幅改善 synthesis estimate，但真正的 setup/hold closure 仍需要實際 placement、routing 與 post-route repair。

表 8.2: Timing 問題從症狀到 final closure 的工程鏈

階段	Setup	Hold	判讀
Historical v3 synthesis	WNS -1.192 ns ; TNS -1506.136 ns ; FEP 1,489	WHS $+0.049$ ns ; FEP 0	Retained historical value ; 主要風險為 1024-bit FF arrays、wide indexed mux 與高 fanout。
v8 synthesis	WNS -0.298 ns ; TNS -11.728 ns ; FEP 292	WHS $+0.045$ ns ; FEP 0	Memory-based UART 與 RTL locality 已大幅減少 violation，但 synthesis interconnect 仍是估計值。
v8 placed + phys_opt	WNS $+0.075$ ns ; TNS 0 ; FEP 0	WHS -0.082 ns ; THS -0.364 ns ; FEP 9	Placement 與 pre-route physical optimization 使 setup 轉正，但尚有 hold paths。
v8c routed strict audit	WNS $+0.029$ ns ; TNS 0 ; FEP 0	WHS $+0.044$ ns ; THS 0 ; FEP 0	Route 與 post-route physical optimization 完成實際 interconnect 與 hold repair；setup/hold 同時 PASS。

Final worst setup path 的 data delay 為 5.544 ns，其中 routing 4.330 ns、約占 78.1%，logic 為 1.214 ns、5 levels。這表示後期瓶頸主要是 physical locality 與 routing，不只是 Boolean logic 太深；因此只改演算法公式或換 route directive 都不足以保證 closure。

8.3 實際採用的 Timing 解法

本設計按下列順序處理 Timing，而不是以一條例外或單一 directive 掩蓋問題：

1. **Memory inference 取代 wide FF arrays**：UART input、data-capture 與 tag-capture 改為 8-bit distributed byte memories，移除大型 indexed mux 與 reset cone。相對 v3，v4 synthesis 減少 10,048 LUT (44.8%) 與 6,434 FF (41.7%)。
2. **建立同步 pipeline boundaries**：shared AES 使用 registered S-box output 與 registered round-key selection，把每個 round 拆成 issue、S-box/key select、ShiftRows/MixColumns/AddRoundKey 三個 pipeline clocks；transform stage 以 valid token 表示資料有效，避免一條高 fanout CE 控制整排 registers。
3. **降低 round-key fanout**：1,920-bit round-key store 的 selector 複製為四份，各自服務一個 32-bit quarter，避免單一 4-bit selector 驅動 128 個 key bits。Key expansion 使用 localized one-hot word write enables，每次只寫一個 32-bit word，不讓 FSM decode 直接驅動整個 store。
4. **FIFO 與 control locality**：FIFO 使用 registered head、局部 full/count logic 與 queue-local clear；ZEROIZE、protocol-abort 與 sensitive payload scrub 維持同步優先序。非敏感 metadata 在 validity 無效時不必全部 reset，縮小 control cone，但 key/plaintext/ciphertext 等敏感資料仍明確清除。
5. **受控 physical replication**：高 fanout reset distribution 可由 Vivado 在 implementation 後複製 physical FF；strict gate 逐份核對來源與 endpoints，確保 replica 仍屬同一 logical control，不引入新 CDC 或不同語意。
6. **完整 implementation flow**：依序執行下列 stages：

```
opt_design -directive ExploreWithRemap
place_design -directive ExtraNetDelay_high
phys_opt_design -directive AggressiveExplore
route_design -directive Explore
phys_opt_design -directive AggressiveExplore
```

證據只支持整個階段前後的改善，不宣稱某一條 directive 單獨必然修復所有 paths。

7. **約束與 hard gates**：只允許真實 asynchronous board I/O exceptions；internal unconstrained endpoints、combinational loops、multiple clocks、unsafe CDC、route errors、DRC Error/-Critical Warning 與 methodology Error/Critical Warning 都必須為 0。

8.4 演算法等價實作與硬體微架構改善

本設計沒有減少 AES rounds、改變 S-box、放寬 authentication tag 規則或修改 GHASH recurrence。因此「演算法改善」在本報告中是指保持 FIPS-197 / SP 800-38D 結果完全等價的

運算結構改善，以及硬體微架構與排程改善。

表 8.3: 演算法等價實作與微架構改善

改善項目	分類	實作、效益與取捨
Hybrid S-box	等價運算 / pipeline	AES S-box 使用完整 256-entry lookup。Shared engine 與 key expansion 使用 registered S-box，規則延遲並利於 BRAM inference；兩個 first-block engines 只在 startup 使用 16-way combinational S-box，以額外 LUT 換取一 round/clock。沒有使用 composite-field inversion。
GHASH16	等價有限體乘法	每拍處理 16 bits，128-bit multiply 為 8 digit clocks，external request II 為 9。 $T^0(V) \dots T^{16}(V)$ 直接從原始 V 形成 closed-form transforms，16 個 masked terms 以 $16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ balanced XOR tree reduction，把 XOR depth 限制為四層。175 MHz raw capacity 約 2.489 Gbit/s，已高於 8-bit stream 的 1.4 Gbit/s 上限，因此沒有採用 Karatsuba 或四路 GHASH。
Shared AES 三 contexts	微架構 / 排程	16 個 registered S-box 共用一條 round datapath，三份 state contexts 交錯執行。單筆 nominal latency 約 30/36/42 clocks，但 steady-state block II 降為 10/12/14 clocks；finish-now lookahead 允許 context 完成的同一 edge 接受替代 request。
兩個 first-block engines	局部平行化	CTR block index 0、1 各由一個 dedicated one-round-per-clock engine 處理，latency 為 10/12/14 clocks；後續 blocks 仍由 shared engine 執行。這是消除 record startup bubble 的 specialization，不是三條完整 AES-GCM lanes。
Key reuse、prefetch 與 queues	排程 / 解耦	Key expansion 在 record 前完成並保存 round keys，後續 records 重用。CTR AES ahead-of-stream 產生 keystream，16-entry FIFO 保持 block order；兩筆 128-bit sensitive data queue 允許 byte admission 與 first-result latency overlap，並支援同 edge enqueue/dequeue。代價是額外 storage、control 與 security verification。

8.5 吞吐率改善、面積與 security 取捨

舊 175 MHz 架構雖然 timing 可通過，但 first-block keystream startup bubble 使六模式低於 1 Gbit/s。加入兩個 first-block engines 與兩筆 data queue 後，改善的是**每個 clock 的有效工作比例**，clock 本身仍為 175 MHz。

表 8.4: 175 MHz 吞吐率改善前後比較

Key mode	舊 Encrypt / Decrypt (Gbit/s)	新 Encrypt / Decrypt (Gbit/s)
AES-128	0.947 / 0.936	1.065398 / 1.051705
AES-192	0.918 / 0.908	1.065398 / 1.051705
AES-256	0.891 / 0.881	1.065398 / 1.051705

依保留到小數第三位的舊結果估算，AES-128/192/256 的 Encrypt 提升約 12.5%/16.1%/19.6%，Decrypt 提升約 12.4%/15.8%/19.4%。

新版本以 100 筆、每筆 1024-bit payload 測得 encrypt 16,820 cycles、decrypt 17,039 cycles，三種 key sizes 全部通過 > 1 Gbit/s。這是 175 MHz RTL/AXI payload throughput，不是 1 GHz core clock，也不是 115200-baud UART 的實板傳輸率。

吞吐量平行化並非免費：Round55 candidate 相較先前 175 MHz source 增加約 3,230 LUT 與 1,046 FF，RAMB18 維持 10。這和前面 UART byte-memory refactor 的面積降低是兩個不同階段，不能宣稱 first-block parallelization 同時降低面積。Final full-board 使用 12,411 LUT、9,029 FF、10 RAMB18、DSP 0，仍在 XC7A100T 容量內。

額外的 queue 可能保存 plaintext/ciphertext，因此 security semantics 沒有因 throughput 而放寬：protocol abort 或 ZEROIZE 會同步清除 sensitive queue payload；decrypt plaintext 先留在兩個 holding banks，只有 tag comparison 完成且 authentication 成功後才進入 release queue。Round48/49/53、fast-zeroize、data-buffer clearing、UVM scoreboard、SVA 與 NIST regressions 都用來防止 stale payload、unauthenticated plaintext 或 queue ordering regression。

8.6 最終驗證與可重現性界線

Final v8c strict audit 的 WNS 為 +0.029 ns、TNS 0、setup FEP 0；WHS +0.044 ns、THS 0、hold FEP 0；WPWS +1.607 ns、TPWS 0、pulse-width FEP 0。17,758 個 routable nets 全部 fully routed，route error 與 DRC violation 為 0，unsafe CDC category 為 0，check_timing critical categories 全為 0。Dynamic side 另以六模式 throughput、SVA/UVM、security regressions、FPGA smoke 與 47,250 筆 NIST RSP 驗證 cycle、資料、tag、status 與清除行為。

Setup margin 只有 29 ps，屬於窄但為正的 margin；結論只對目前 exact RTL/XDC、Vivado 2021.1 build、implementation directives 與 SHA-matched DCP 成立。更換任何一項都必須重新執行 synthesis、place、phys_opt、route 與 strict audit。這也是選擇同步設計的主要工程價值：每個 register-to-register path、CDC exception 與 reset topology 都能被明確量測、約束及回歸。

第九章 Wrapper、UART 封包與傳輸效率

9.1 Production AXI IP wrapper：設計目標與責任邊界

`aes_gcm_axi_top.v` 是正式系統整合所使用的 production-facing wrapper；它和外層 `arty_a7_100t_aes_gcm_uart_rsp_top.v` 的角色不同。前者把 `aes_gcm_stream_core` 封裝成 AXI4-Lite 加 AXI4-Stream IP，後者只是在 Arty A7 上以 UART 重播 NIST 向量的 board-validation transport。這一層 Wrapper 不執行 AES round 或 GHASH，而是集中承擔下列三種系統責任：

表 9.1: Production AXI wrapper 的三個責任平面

責任平面	主要 RTL 結構	對外保證	設計理由
Control plane	<code>u_registers</code> 、register bank、one-cycle command pulses、admission equations	AXI4-Lite write/read、可觀測 status、非法或當下不可接受的命令以 <code>SLVERR</code> 拒絕且不產生 side effect	將軟體設定、core lifecycle 與命令原子性集中在單一邊界
Streaming data plane	2-entry input queue、one-entry Data skid、2-entry Tag FIFO、Result gate	只有成功接受 descriptor 後才能送 input frame；一般 backpressure 下公開 payload 穩定；避免 Data/-Tag/Result 互相超車	讓 byte transport、core acceptance 與 downstream stall 以局部 buffer 解耦
Security / ordering boundary	input-frame credits、outermost frame trackers、ZEROIZE masks/clears	authentication-before-release、mode-dependent completion order、emergency ZEROIZE flush	防止未認證 plaintext、stale frame 與跨 transaction 公開輸出混線

因此「Wrapper 只是接線」是不正確的描述。較精確的設計說法是：**Core 負責密碼運算與 record lifecycle；Wrapper 負責把軟體命令、公開 byte stream、backpressure、frame ownership 與 emergency scrub policy 轉換成 Core 可以安全接受的同步事件。**

它是手寫 RTL，目前 repository 沒有 `.xci`，也不是 Xilinx IP Integrator 自動產生的 packaged IP。若後續要放入 block design，可再建立 IP-XACT component packaging 與 driver header；這不改變目前 RTL、synthesis、implementation 與 board evidence 的有效性。

9.1.1 AXI4-Lite control plane 與 register map

`u_registers/axi_lite_regs` 是 32-bit、one-outstanding-transaction AXI4-Lite register bank。AW 與 W channel 可分開握手並各自保存；只有兩者都已捕捉、且 B channel 沒有尚未完成的 response 時，才在一個 active edge dispatch 該 write。CONTROL write 成功後所產生的 `cmd_push_pulse`、`key_commit_pulse` 或 `zeroize_pulse` 都只有一個 clock；即使 master 暫時不拉高 `BREADY`，pulse 也不會因 `BVALID` stall 而重複。

表 9.2: Production AES-GCM AXI wrapper 的精確 AXI4-Lite register map

位址	名稱 / 存取	欄位	設計語意
0x00	CONTROL / W pulse	bit0 CMD_PUSH; bit1 KEY_COMMIT; bit2 ZEROIZE	只有 byte lane 0 的 <code>WSTRB[0]</code> 有效；一次只能要求一個 command。多個 command bits 同時為 1 時整筆 write 以 <code>SLVERR</code> 拒絕。全 0 或未選 lane 0 是 OKAY no-op。
0x04	STATUS / R	bit0 KEY_READY; bit1 STATUS 在 AR handshake 取 snap- KEY_BUSY; bit2 CMD_FULL; shot, 再經 pending stage 送出；若 AR bit3 RECORD_ACTIVE; bit4 和 command dispatch 發生在同一 edge。 OUTPUT_PENDING; bit5 ZE- ROIZE_BUSY	讀值可能反映 command 前的狀態。
0x08	CFG / R/W	bit0 DECRYPT; bits15:8 TAG_BITS	這不是 AES key-mode register。key_mode[2:0] 是 Wrapper 的獨立 input，於 KEY_COMMIT pulse 一起被 Core 取樣。
0x0C	IV_LEN / R/W	bits10:0，單位為 bit	允許最多 2,047 bits；CMD_PUSH 時另外檢查不可為 0。
0x10	AAD_LEN / R/W	bits10:0，單位為 bit	0 合法；實際 byte ordering 由 input frame 與 descriptor 長度共同定義。
0x14	DATA_LEN / R/W	bits10:0，單位為 bit	0 合法；encrypt/decrypt 共用此長度欄位。
0x20--0x3C	KEY_WORD0-7 / W	8 個 32-bit staging words	WORD0 寫入 key[255:224]，WORD7 寫入 key[31:0]；每個 byte 依 <code>WSTRB</code> 更新， <code>WDATA[31:24]</code> 是該 word 最早的 AES byte；architectural read 回 0。
0x40	CAPABILITY / R	version、key modes、max length、tag policy	default 為 0x0107B7FF；若啟用 NIST truncated-tag build 則為 0x0107BFFF。

Reset 後 `DECRYPT=0`、`TAG_BITS=128`、`IV/AAD/DATA` lengths 為 0、key staging 為 0。ZEROIZE 會把 key staging 清為 0，但不會改寫 CFG 與 length registers；這使 software 可以保留非敏感的 descriptor configuration，同時讓下一把 key 必須重新載入。

9.1.2 三條命令不是同一組 admission checks

漂亮的概念圖很容易把 KEY_COMMIT、CMD_PUSH 與 ZEROIZE 畫成共用一個 checklist；exact RTL 並非如此。三者有不同的安全目的，因此使用三種不同規則：

1. **CMD_PUSH**：建立一筆 record descriptor。接受條件為 key 已 ready、descriptor FIFO 未滿、Core 不在 ZEROIZE、IV length 不為 0、tag length 符合本 build policy。它刻意**不要求**目前 record 或 public output 完全 idle，只要 descriptor FIFO 有空間，就能讓下一筆 descriptor 預先排隊。
2. **KEY_COMMIT**：把 256-bit staging key 與外部 key_mode 交給 key expansion。接受條件為 key_mode 只可為 0/1/2 (AES-128/192/256)、key engine 不 busy、Core key lifecycle quiescent、Data skid 與 Tag FIFO 沒有 boundary residue、Core 不在 ZEROIZE。它不使用 IV/tag length 或 CMD_FULL 作為條件。
3. **ZEROIZE**：standalone、格式合法的 ZEROIZE write 不依賴上述 lifecycle 狀態而被 state-reject；它是 emergency flush，不是「等輸出 drain 完再清除」。但若同一 CONTROL write 同時設了其他 command bit，仍因 command 不具原子唯一性而回 SLVERR。

Default production build 只接受 128-bit tag；當 ALLOW_NIST_TAG_LENGTHS=1 時，另外接受 32、64、96、104、112 與 120 bits。CFG write 本身只保存值並回 OKAY，真正的 tag-policy validation 延後到 CMD_PUSH；這使設定階段與 descriptor admission 的錯誤責任清楚分離。任何被拒絕的 CMD_PUSH/KEY_COMMIT 都只回 SLVERR、不產生 pulse、也不改變 Core；目前沒有 sticky error register，因此 software 必須檢查每一筆 B response。

9.1.3 Input plane：2-entry registered-head queue 與 frame credits

公開 input 不是直接接到 Core，而先進入一個 2-entry、9-bit FIFO；每一 entry 保存 1-bit TLAST 與 8-bit TDATA[7:0]。這個 queue 的設計目的有兩個：第一，用 registered head 暫存 byte；第二，以 registered s_axis_tready_r 切斷複雜的 Core byte-acceptance logic 到 public READY 的組合路徑。

input_frame_credits 不是 FIFO occupancy，也不是整筆 transaction 的 lifetime counter。它精確表示「已成功 CMD_PUSH、但 public input TLAST 尚未 handshake 的 frame 數量」：

- 成功的 CMD_PUSH 增加一個 credit；
- public TVALID && TREADY && TLAST 減少一個 credit；
- 同一 edge 同時 CMD_PUSH 與 TLAST handshake 時淨變化為 0；
- 沒有 credit 時 public READY 一定為 0，因此 software 不可能在 descriptor 被接受以前塞入資料；
- early TLAST 會先在 public boundary 歸還 credit，Core 之後再回報 framing abort；late TLAST 則保留 credit，直到 drain 過程真的收到後續 TLAST。

READY 使用 predictive occupancy：每個 edge 根據本拍 push/pop 後的 queue count 與下一拍 credit 狀態，決定下一 cycle 是否仍可接資料。Reset 後 READY 為 0；CMD_PUSH 被取樣後，credit 與 registered READY 在 edge 更新，最快下一個 edge 才可能接收第一個 byte。當 queue 同時 pop 與 push 時可維持每 clock 一 byte；若無 pop 且本拍接收使預測 count 變成 2，下一 cycle READY 會先行拉低，避免 overflow。

9.1.4 Output plane : elasticity 、outermost frame ownership 與 mode-dependent ordering

三條 output 的 buffer 並不相同，因此不能統稱為「都用 registered READY 完全隔離」：

表 9.3: 三條公開 output 的實際 buffering 與 timing 特性

Channel	RTL 結構	Backpressure 行為	必須誠實說明的 timing 邊界
Data	one-entry axis_output_skid_8	普通 stall 時 DATA/LAST/USER 穩定；空時可先收一拍；同一 edge 可送舊 beat 並補新 beat	它提供 forward payload register 與 one-beat elasticity，但 m_axis_data_tready 在 skid occupied 時仍可組合影響 core_data_tready，所以不是完整 READY register slice
Tag	2-entry、9-bit registered-head FIFO	head 在 stall 時穩定；Core READY 取決於 registered count；可在 Data 尚未送完時先 buffer tag bytes	這條 path 才具有 registered-occupancy READY isolation；full 後 pop 的 refill 仍有一拍限制，第二個 entry 用來隱藏 public output bubble
Result	無 FIFO / 無 skid；ordering gate 直接連 Core result register	Core 必須在 public stall 時保持 VALID/data；只有 boundary trackers 判定可離開時才傳遞 READY	不提供額外 buffering；它解決的是 ordering eligibility，而非 backpressure decoupling

Wrapper 不能用「目前 FIFO 是否空」判斷 frame 結束，因為 authenticated plaintext 可能在 16-byte BRAM block loading 之間出現合法 bubble。設計因此使用 data_boundary_idle_r 與 tag_boundary_idle_r 追蹤最外層公開 handshake：接受任何一個 Core Data/Tag beat 後即標記 frame busy；只有 public final TLAST 真正 handshake 才回 idle。public_frames_idle_core_r 再把兩個 predicted next states 註冊起來，使 Result 最早在最後一個 Data/Tag handshake 的下一 cycle 可見，不會在同一 edge 穿越 boundary。

真正的公開順序必須依 mode 說明：

表 9.4: 合法完成與錯誤路徑的公開輸出順序

情境	公開順序	安全 / 協定意義
Encrypt success	Ciphertext Data frame → generated Tag frame → ENC_OK Result	Result 不能超過尚未完成的 Data 或 Tag
Decrypt authentication success	DEC_AUTH_OK Result → authenticated Plaintext Data frame	Plaintext 先留在 protected banks ; 只有 success Result 被接受後, Core 才啟動 release
Decrypt authentication failure	AUTH_FAIL Result only	不產生任何未授權 plaintext
Protocol abort after visible encryption prefix	已 buffer 的 prefix → synthetic zero/TLAST terminator → abort Result	先合法關閉已開始的 public Data frame, 再回錯誤狀態
ZEROIZE	撤銷尚未 handshake 的 Data/Tag ; scrub 完 成後輸出仍欠的 ZEROIZE_ABORT Results	emergency flush ; 不保證把 partial frame drain 或補 terminator

所以正確保證不是「永遠 Data、Tag、Result」, 而是: **Result 不得超越已開啟或已 buffer 的 public Data/Tag frame ; decrypt plaintext 另受 authentication-before-release 規則控制。**

9.1.5 ZEROIZE : emergency flush, 而不是 graceful drain

ZEROIZE pulse 是 AXI register bank 完成 buffered CONTROL write dispatch 後產生的 synchronous one-cycle event。Pulse 為 1 的 cycle, Wrapper 立即將 public input READY 與所有 public output VALID gate 為 0 ; 在取樣 pulse 的 active edge, input FIFO、Data skid、Tag FIFO、input credits 與 boundary trackers 都被 synchronous clear/reset, key staging 清為 0, 同一 pulse 直接送進 Core 啟動 sensitive-state invalidation 與 plaintext-bank scrub。

這個定義有四個必須明說的界線:

1. 已經完成 public handshake 的 byte 無法被撤回 ; ZEROIZE 只能阻止尚未 handshake 的公開資料。
2. 這是 flush semantics。被 stall 的 VALID 可以被撤銷, partial Data/Tag frame 可以被截斷, ZEROIZE 不為它補 synthetic TLAST。
3. generic FIFO 的 clear 會清 pointers、count、registered head 與 validity, 使 stale entry 不再可觀察 ; 它不是對每個 backing mem[] word 逐位物理抹除。若產品威脅模型要求 memory remanence scrub, 必須另加 explicit overwrite 或使用具有 erase guarantee 的 storage policy。
4. Core 只對「Result 尚未退休」的 unfinished descriptors 保留對應的 ZEROIZE_ABORT 債務。若 DEC_AUTH_OK 已 handshake、但 authenticated plaintext 尚在 release path, ZEROIZE 可丟棄剩餘 plaintext 而不再產生第二個 abort result ; 這和 emergency flush 定義一致, 但不能宣稱每一個尚未輸完的 byte 都會得到替代 Result。

目前 Wrapper 連接但不採用 `core_zeroize_defer`，因此 policy 明確偏向立即 flush。若未來產品規格改成「先保存 / drain 已認證輸出，再 scrub」，就不能只修改文件；必須把 release-producer、allocated plaintext bank、boundary buffer 與 repeated ZEROIZE 都納入 defer contract，再以 SVA 和 directed fault injection 重新驗證。

9.1.6 Protocol abort 與 ZEROIZE 的差異

TLAST protocol error 是單一 record 的協定錯誤，不應破壞已排隊的其他 descriptor。Early TLAST 進入 abort wait；late TLAST 進入 drain 並丟棄 byte，直到收到後續 accepted TLAST。若 encrypt 已有 prefix 跨越 Core boundary，Core 會送 synthetic zero/TLAST terminator，再送 abort Result。這條路徑不會像 ZEROIZE 一樣清掉 top input queue、Data skid、Tag FIFO、descriptor FIFO 與舊的完成結果。

Core 內部在 abort flush 時可以更換尚未被 Core-output handshake 接受的 stalled beat；public boundary 的 Data skid 會保護已被 Wrapper 接受的公開 beat，使外部一般 AXI stability 仍成立。因此若未來有人直接繞過 `aes_gcm_axi_top` 使用 `aes_gcm_stream_core`，就必須自己加等價 output buffering，不能假設 Core raw interface 已具有相同 public guarantee。

9.1.7 設計取舍、可驗證性與報告結論

此 Wrapper 選擇的不是「每一條 channel 都加很深 FIFO」，而是針對問題配置最小結構：input 需要 registered READY 與 command credit，因此用 2-entry queue；Data 需要一拍 elasticity 與 payload hold，因此用 one-entry skid；Tag 需要在 Data 尚未結束時提前吸收 byte，因此用 2-entry FIFO；Result 只需要防超車，因此使用 frame gate 而不加 FIFO。代價是 ordering logic 與 mode semantics 較複雜，但可避免不必要的 area/latency，並讓每個 correctness property 有清楚的 checker boundary。

報告這一頁時可用一句話收斂：

我把 AXI Wrapper 設計成同步的 protocol/security boundary：control plane 將 software write 原子化為不同 admission 規則的命令事件；input queue 與 output buffers 管理 byte-level backpressure；outermost frame trackers 保證公開順序；ZEROIZE 則採立即 flush 並觸發 Core scrub。它的目的不是做 AES-GCM 運算，而是讓同一個 Core 能以可整合、可 timing、可驗證的方式進入真正系統。

主要 code anchors 為 `aes_gcm_axi_top.v:94--100` (admission)、`aes_gcm_axi_top.v:139--211` (input queue / credits / registered READY)、`aes_gcm_axi_top.v:280--378` (public frame ownership / outputs) 與 `stream_axi/axi_lite_regs.v:127--290` (CONTROL dispatch / register write semantics)。Vivado 原生 schematic 可證明 `u_registers`、`u_input_queue`、`u_core`、`u_data_output_register` 與 `u_tag_output_register` 的 hierarchy/connectivity；UVM/SVA/NIST 則分別證明 transaction result、cycle-level invariant 與標準向量 correctness，三者不可互相取代。

9.2 UART board wrapper

arty_a7_100t_aes_gcm_uart_rsp_top.v 使用：

- Xilinx MMCME2_BASE 與 BUFG primitives 產生 175 MHz；
- mmcm_locked \&\& rst_btn 先進入 2-stage ASYNC_REG synchronizer，再經 4-stage stable-high qualifier 產生 synchronous aresetn；
- 手寫 uart_rx.v 與 uart_tx.v，不是 Xilinx AXI UART IP；
- aes_gcm_uart_rsp_bridge.v 將 UART frames 轉成 AXI4-Lite writes 與 AXI4-Stream data；
- LED 顯示 heartbeat、busy、last pass、last fail/MMCM unlock。

Reset directed XSim test 拒絕 1–3 個 sampled clocks 的 button/LOCKED pulse 與 alternating chatter；continuous-valid input 在第 6 個 sampled edge 釋放 reset，invalid input 在第 3 個 sampled edge 讓 distributed reset 生效。Routed audit 另以結構 gate 確認 logical qualifier 有 4 stages；最後一階為了 physical fanout 複製成 4 個 cells，不是 4 個獨立邏輯 reset sources。CDC 報告的 2 條 async port path 使用 false path 並由 synchronizer 處理；downstream FF-to-FF stages 仍由 STA 檢查。

Host request frame 格式為

$$\text{command}[7:0] \parallel \text{bit_length}[15:8] \parallel \text{bit_length}[7:0] \parallel \text{payload}. \quad (9.1)$$

Commands 為 MODE、KEY、IV、AAD、PT、CT、TAG、RUN；response frames 為 STATUS、DATA、TAG。Bridge 將 input payload 存於 8-bit distributed RAM，output data/tag 也先捕捉到 byte memories，再依 frame 回傳。這個 memory-based wrapper 是 board timing closure 的關鍵。

9.3 Protocol efficiency 與錯誤處理

UART 8N1 的 line coding efficiency 固定為 80%；再乘上 $N/(N+3)$ 的 frame header efficiency。Bridge 檢查 key length、IV/AAD/data 最大 1024 bits、NIST tag lengths、busy state、AXI response、result watchdog 與 output length/TLAST/TUSER。錯誤以 BAD_CFG、BUSY 或 AUTH_FAIL 回應。Protocol simulation 已涵蓋 AES-128/192/256 encryption、decrypt success、decrypt auth-fail 與 data block，共 6/6 pass。

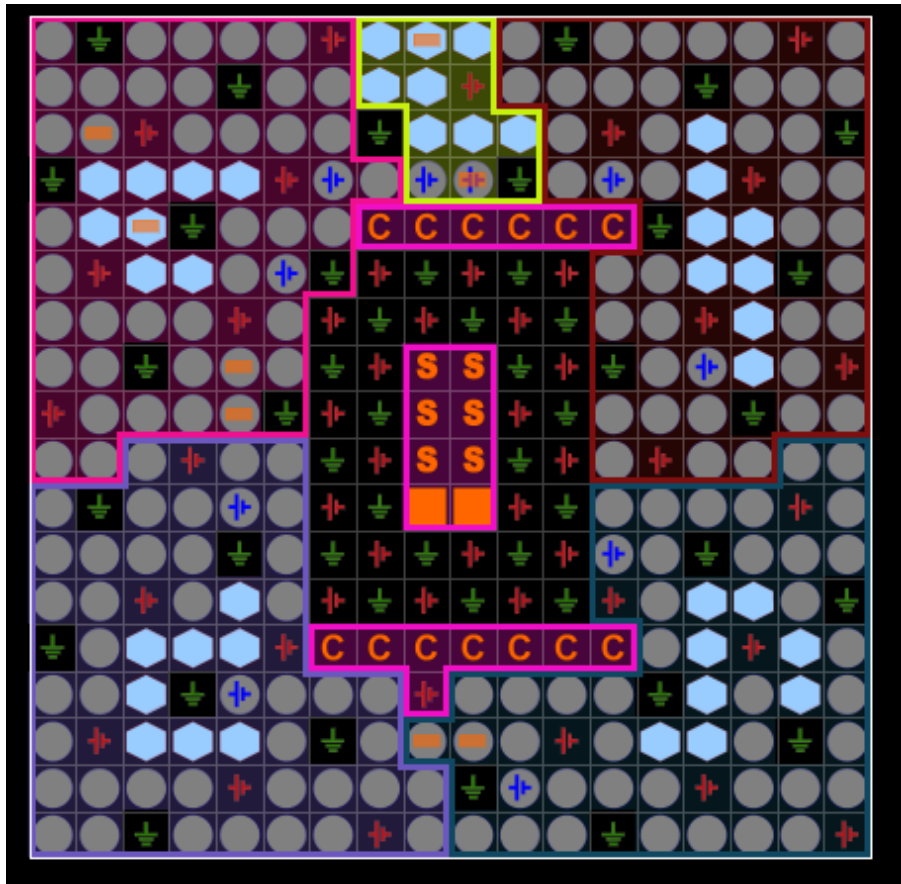
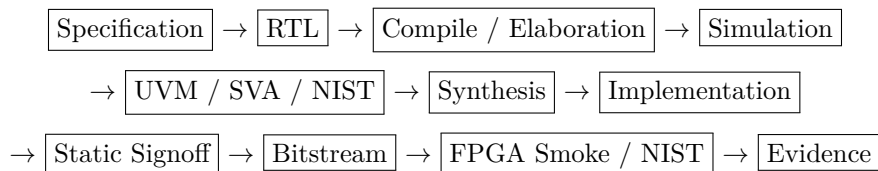


图 9.1: Vivado 原生 Package / I/O 视图的清晰裁切；显示 XC7A100T CSG324 package 的电源、时脉与 I/O pins，供 board-level pin planning 查核。DRC 的 No Violations Found 证据另见图 7.6。

第十章 驗證策略、NIST 與 FPGA 實板結果

10.1 端到端設計與驗證流程

本專案採用 stage-gated flow：每一階段都有明確輸入、工具、checker、PASS criteria 與可保存 artifact；前一階段成功不會自動代表下一階段成功。例如 RTL simulation 通過只能證明已執行向量下的功能，synthesis 成功只能證明 RTL 可映射成 FPGA primitives，真正的 175 MHz timing closure 必須由 routed static timing analysis 判定，而板級正確性仍需 exact bitstream 的 hardware tests。完整流程為



10.1.1 Stage-gated verification plan

表 10.1: 從規格、RTL 到 FPGA 實板的端到端驗證流程

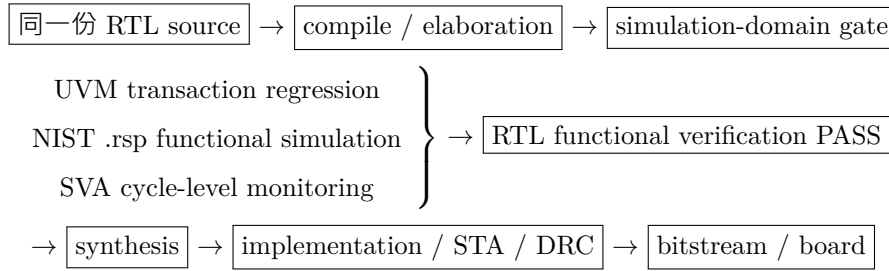
Stage	工作內容	工具與主要 artifact	PASS gate 與本版結果
1	設計規格與 requirement decomposition	FIPS-197、NIST SP 800-38D、AXI/UART protocol、175 MHz 與 security requirements	AES-128/192/256、encrypt/decrypt、> 1 Gbit/s、authentication-before-release、ZEROIZE、setup/hold/legality gates 都有 checker 對應。
2	RTL implementation 與 compile/elaboration	SystemVerilog RTL；xvlog、xelab；elaborated hierarchy 與 compile log	Production source list 可編譯、top hierarchy 與 bind schema 存在，無 compile/elaboration error。
3	RTL smoke 與 directed regression	XSim；GHASH standalone、AXI smoke、Round48/49/53、FIFO clear/refill、reset qualifier tests	GHASH 66 tests、AXI smoke 7,676 cycles、security/reset directed suites 通過，任何 Assertion failed 都使 runner non-zero。
4	Transaction-level UVM	XSim UVM 1.2；virtual sequence、AXI agents、passive reconstruction、independent reference model、scoreboard、coverage DB	Seeds 175001/175019/175039，各 8 scenarios；24 checked、0 failed、UVM E/F 0/0、maintained functional coverage 100%。

Stage	工作內容	工具與主要 artifact	PASS gate 與本版結果
5	Cycle-level SVA 與 full simulated NIST	37 source assertions 由 <code>bind_all.sv</code> 綁入；六份 NIST RSP 的 XSim testbench	Retained traces 中 assertion failure 0；simulation NIST 47,250/47,250、fail 0、17,740,527 cycles。
6	Synthesis	Vivado <code>synth_design</code> ；synth DCP、timing 與 utilization reports	RTL 成功映射為 LUT/FF/LUTRAM/BRAM/MMCM/BUFG。Synth timing 只作 early indicator，不作 final closure 判定。
7	Placement、physical optimization 與 routing	<code>opt_design</code> 、 <code>place_design</code> 、 <code>phys_opt_design</code> 、 <code>route_design</code> ；placed/routed DCP	Final routed WNS +0.029 ns、TNS 0、WHS +0.044 ns、THS 0、setup/hold FEP 0。
8	Static implementation hard gates	Vivado timing、pulse-width、route、DRC、methodology、CDC、 <code>check_timing</code> reports	17,758/17,758 routable nets 完成；route error、DRC E/CW、methodology E/CW、unsafe CDC 與 10 個 critical timing categories 均為 0。
9	Bitstream 與 FPGA smoke	Hardware Manager、v8a <code>program_board.tcl</code> 、UART host runner、bitstream SHA-256	exact bitstream 成功程式化到 <code>xc7a100t_0</code> ；AES-128/192/256 encryption/decrypt 為 6/0。
10	Hardware NIST 與 evidence reconciliation	Python + <code>pyserial</code> ；COM4/115200；JSONL、summary、RSP 與 bitstream hashes	47,250/47,250、fail 0；11,908 expected authentication failures；status/data/tag violations 0；second reconciliation 亦通過。

本流程沒有把 gate-level timing simulation 當作 routed timing signoff。Internal setup/hold、clock pulse width、routing completeness 與 CDC legality 由 Vivado static analysis 處理；RTL/UVM/NIST simulation 負責功能與 protocol；FPGA UART runner 負責真實 configuration image、wrapper 與 board I/O 的 operational evidence。三者互補，不能互相取代。

10.1.2 RTL 模擬 gate：UVM、NIST 與 SVA 的先後關係

UVM、NIST RSP 與 SVA 都在 synthesis 之前檢查同一份 SystemVerilog RTL，但三者不是同一個 testbench，也不應與 FPGA board result 合併解讀。實際執行順序是「RTL compile/elaboration → simulation-domain verification gate → synthesis → implementation/signoff → exact-bitstream board replay」。只有 simulation gate 通過，RTL 才進入 synthesis；只有 fully routed design 的 timing、route 與 DRC hard gates 通過，才可產生 promotion bitstream；最後才由 FPGA board 驗證真實 wrapper、clock、UART 與 configuration image。



模擬路徑	Stimulus 與入口	Checker / oracle boundary	Retained PASS evidence
UVM 1.2	tb_uvm_top.sv ; virtual sequence 經 AXI-Lite 與 AXI-Stream agents 驅動 DUT	Passive reconstruction monitor 只根據實際 handshakes 重建 transaction ; independent AES-GCM model 與 scoreboard 比對 data 、tag 、result 、authentication outcome	3 seeds 、每 seed 8 scenarios ; 24 checked 、0 failed 、UVM E/F 0/0 ; maintained scenario coverage 100%
Full NIST RSP	tb_nist.sv 讀取 6 份 RSP sources ; AES-128/192/256 · encrypt/decrypt	Testbench 逐筆比對 plaintext/ciphertext 、tag 與 expected FAIL ; 此路徑不是 UVM sequence 或 UVM scoreboard	47,250/47,250 PASS ; fail 0 ; 17,740,527 simulation cycles
SVA	bind_all.sv 將 checker 綁到 GHASH 、key context 、stream core 與 AXI outputs ; 隨 smoke/NIST trace 被動取樣	逐 clock edge 檢查 handshake 、queue 、ZEROIZE 、tag serializer 與 GHASH recurrence ; 不產生 transaction expected value	37 source assertions 、39 active instances ; retained simulation traces 中 assertion failure 0

表 10.2: Simulation domain 內三條獨立 checker path ; full RSP path 為 **tb_nist** + bound SVA · UVM regression 另行執行。

因此，「UVM PASS」代表 transaction-level checker 已通過；「NIST 47,250 PASS」代表 full RSP simulation 的 expected/actual comparison 通過；「SVA failure 0」代表 retained traces 中未違反已綁定的 temporal properties。這三項都不等於 synthesis 或 implementation timing closure，也不能單獨證明實板；它們共同構成 RTL 進入 synthesis 前的 functional verification gate。

10.1.3 UVM 如何從訊號走到 scoreboard

UVM environment 由 AXI-Lite agent、AXI-Stream input agent、三個 output agents、virtual sequencer、passive record reconstruction monitor、scoreboard 與 coverage subscriber 組成。Sequence 先透過 AXI-Lite 寫入 key、mode、IV/AAD/data/tag lengths 與 command；input driver 逐 byte 驅動 TDATA/TVALID/TLAST，只有 TVALID && TREADY 的取樣 edge 才算 DUT 接受。Output monitors 同樣只在 handshake 時採集 data、tag 與 result，因此 backpressure 期間不會重複計數。

Checker path 刻意與 driver 分離。Passive monitor 不直接重用 sequence item，而是從實際 AW/W、input stream 與 output stream handshakes 重新建立 record。重建後才呼叫 independent AES-GCM model，產生 expected payload、tag 與 authentication predicate，再由 scoreboard 比對：

Step	UVM component	實際資料流與觀察點	FAIL condition
1	Virtual sequence / sequencer	依 scenario 產生 key mode、encrypt/decrypt、IV/AAD/data/tag lengths、payload、received tag、input gaps	Randomization 失敗、scenario 不存在或 sequence timeout
2	AXI-Lite driver	寫入 8 個 key words，觸發 key expansion，輪詢 ready，再寫 mode/lengths 與 start command	AXI response 非 OK、key expansion 超時
3	AXI-Stream input driver	依 IV → AAD → payload → received tag 的順序送 byte；最後一個 byte 送 TLAST	只有 TVALID && TREADY 才算接受；錯誤握手不得改變已接受長度
4	Passive reconstruction monitor	從 AW/W、input、data、tag、result 的真實 handshake 重建 record；不讀取 sequence 的 expected item	orphan output、缺少/重複 result、transaction 未完整即 timeout
5	Independent reference model	依 FIPS-197 AES 與 SP 800-38D GCM 規則計算 expected payload、tag 與 auth_ok	Reference-model self-test 或輸入尺寸契約不成立
6	Scoreboard	Encrypt 比對 ciphertext/tag/result；decrypt success 比對 plaintext/result；decrypt auth-fail 要求 plaintext 不可輸出	Data/tag/status mismatch，或 unauthenticated plaintext 被釋出
7	Coverage / runner gate	收集 key、mode、IV/AAD/data length classes、auth pass/fail 與 crosses；解析 UVM summary	checked 不是 8/seed、failed 非 0、UVM E/F 非 0、required maintained bins 未達 100%

表 10.3: UVM transaction 從 stimulus、interface handshake、passive observation 到 scoreboard 的完整生命週期。

```

if (input_tvalid && input_tready)
    reconstruct_input_byte(input_tdata);

if (output_tvalid && output_tready)
    observed_data.push_back(output_tdata);

aesgcm_reference_model::predict(...,
    expected_data, expected_tag, auth_ok);

if (decrypt && !auth_ok && observed_data.size() != 0)
    `uvm_error("DATA",
        "unauthenticated plaintext was released")

```

UVM class hierarchy 與程式碼對照

上述流程不只是一張概念圖；其實作分散在 testbench top、UVM package、reference model 與 XSim runner 四個 source boundary。tb_uvm_top.sv 實例化 DUT、clock/reset 與 AXI virtual interfaces，並以 uvm_config_db 將 control、input、data/tag/result output interfaces 傳給 environment。aesgcm_uvm_pkg.sv 則以官方 UVM 1.2 base classes 建立本專案的 item、agent、monitor、scoreboard、coverage 與 test；UVM framework 是 XSim 載入的 library，AES-GCM/AXI protocol 行為則完全由本專案 source 定義。

```
tb_uvm_top
-> aesgcm_xsim_random_test : uvm_test
    -> aesgcm_env : uvm_env
        -> AXI-Lite agent      : sequencer + driver + monitor
        -> AXI-Stream input agent: sequencer + driver + monitor
        -> Data/Tag/Result agents: monitor + output-ready driver
        -> passive reconstruction monitor
            -> aesgcm_scoreboard
            -> aesgcm_coverage
```

Source / class	UVM base 或型態	本專案責任
tb_uvm_top.sv	SystemVerilog module	實例化 aes_gcm_axi_top、產生 clock/reset，並以 uvm_config_db 注入 5 組 AXI virtual interface。
aesgcm_record_item	uvm_sequence_item	定義一筆 record 的 key mode、key、encrypt/decrypt、IV/AAD/data/tag lengths、payload、received tag 與 observed outputs。
axi_lite_agent、 axis_input_agent、 axis_output_agent	UVM agent composition	將 protocol-specific sequencer、driver、monitor 封裝；output agent 另由 ready driver 製造已定義的 downstream ready profile。
aesgcm_virtual_sequencer / constrained-random sequence	uvm_sequencer / uvm_sequence	同步排程 control writes、input bytes 與 output-ready behavior；每個 XSim seed 執行 8 個 coverage-oriented scenarios。
passive_record_reconstruction_monitor	uvm_monitor	只從實際 AXI handshakes 重建 record；它不共用 sequence/driver 的 transaction object。
aesgcm_scoreboard	uvm_component	呼叫 independent model，比對 payload、tag、唯一 result 與 authentication outcome；auth-fail plaintext release 為 hard failure。
aesgcm_coverage	uvm_subscriber	對 reconstructed record 取樣 key/mode/IV/AAD/data/auth bins 與指定 crosses。
aesgcm_reference_model_pkg.sv	independent SV package	以獨立 AES/GHASH implementation 計算 expected data、tag 與 auth_ok，不重用 DUT lookup table 或 digit-serial GHASH datapath。

表 10.4: XSim UVM 1.2 驗證環境的 source/class 分工；UVM library 提供共通 framework，本專案實作 AES-GCM/AXI 驗證語意。

Environment 的 `connect\_phase` 是此架構的關鍵：virtual sequencer 取得各 agent 的 sequencer handle；同一個 reconstructed record 再以 analysis port 同時廣播給 scoreboard 與 coverage，而不是由 driver 把預期答案傳入 checker：

```
virtual_sequencer.axi_lite_seqr = axi_lite.sequencer;
virtual_sequencer.input_seqr    = input_axis.sequencer;
virtual_sequencer.data_seqr     = data_output.sequencer;
virtual_sequencer.tag_seqr      = tag_output.sequencer;
virtual_sequencer.result_seqr   = result_output.sequencer;

reconstruction.ap.connect(scoreboard.record_imp);
reconstruction.ap.connect(coverage.analysis_export);
```

此連接方式形成「driver stimulus $\neq$ checker observation」的驗證隔離：若 DUT 在 backpressure 下漏送、重複送、錯序送出 data/tag/result，monitor 會重建出與 stimulus 不同的 observed record，scoreboard 即會報出 mismatch。這也是 UVM path 能發現 AXI transaction-level integration error，而不只檢查單一 AES block 演算法的原因。

Constrained-random item 包含 key mode、key、encrypt/decrypt、IV/AAD/data/tag bit lengths、payload 與 received tag。三個 XSim seeds 各執行固定的 8 類 coverage-oriented scenarios，涵蓋 AES-128/192/256、encrypt/decrypt、96/non-96-bit IV、empty/short/one-block/multi-block AAD/-data 與一個 corrupted-tag negative case。Runner 同時檢查 scoreboard count/failure、全域 UVM error/fatal 與 maintained functional coverage bins，任何一項不符即 FAIL。ModelSim Starter 10.5b 的責任邊界不同：它只執行一筆 deterministic AES-128 UVM smoke，結果為 scoreboard 1/0、UVM E/F 0/0。Starter 環境沒有完成本 regression 所需的 constrained randomization 與 covergroups，所以狀態明確保留為 constrained-random NOT RUN、coverage NOT COLLECTED；完整 random 與 functional coverage 證據來自 XSim。

10.1.4 SVA、UVM、Vivado 與實板 checker 的分工

UVM 適合檢查一筆完整 transaction 的 end-to-end data、tag、status 與 authentication outcome；SVA 則以 clock edge 為單位檢查局部 protocol 與 control invariants。例如 AXI output 在 VALID=1、READY=0 時，下一拍 valid 與 payload 必須保持；ZEROIZE 後 round keys 與 queue 必須在規定 edge 清除：

```
assert property (disable iff (!rst_n || clear)
    (valid && !ready) | =>
    (valid && $stable(payload)));

assert property (disable iff (!rst_n)
    zeroize | =>
    (!ready && !busy &&
    round_count == 0 && round_keys == 0));
```

SVA 透過 bind 掛到 ghash16、aes_key_context、aes_gcm_stream_core 與 aes_gcm_axi_top，不修改 production RTL，也不合成進 bitstream。Simulation runner 把 RTL 與 SVA 一起交給

xvlog、xelab 與 xsim；任何 assertion failure 都會使 smoke 或 NIST suite 失敗。現有 37 條 source assertions 在 retained tests 中 failure 為 0，但沒有 assertion activation/vacuity coverage，因此不能宣稱 100% assertion coverage。

SVA 階段	工具 / artifact	驗證意義與 hard-fail 行為
Compile	xvlog -sv 編譯 production RTL、5 個 checker source 與 bind_all.sv	Checker 與 DUT port/state schema 必須一致；語法、名稱或 binding 錯誤使 suite 停止。
Elaboration	xelab 展開 module hierarchy 與 bind instances	37 條 source assertions 依實際 module instance 展開為 39 active instances；production RTL 不需插入 checker code。
Runtime sampling	xsim 在 clock edge 評估 ->、 =>、\$past、\$stable 與 disable iff	監看 GHASH recurrence、key lifecycle、queue/serializer progress、AXI backpressure stability 與 ZEROIZE 清除規則。
Failure propagation	Smoke/NIST runner 搜尋 Assertion failed 並以 non-zero 終止	Property 名稱與失敗 cycle 可定位局部 control/protocol 錯誤；即使 final ciphertext 碰巧相同，temporal violation 仍會擋下 promotion。
Evidence boundary	Retained smoke/NIST logs 與 summary	failure 0 只代表已執行 traces 未觸發違規；未收集 activation、vacuity 或 assertion coverage，因此不宣稱 exhaustive proof 或 100% assertion coverage。

表 10.5: SVA 從編譯、bind/elaboration、逐拍監看到 runner hard gate 的執行流程。

Vivado implementation 則檢查 simulation 看不到的 physical 問題：真實 routing delay、setup/hold、clock pulse width、unrouted nets、DRC、methodology、CDC schema 與 check_timing。只有 routed hard gates 全部通過才產生可 promotion bitstream。板上驗證再用 exact bitstream hash 綁定六模式 smoke 與 NIST RSP；host 逐筆比對 status、ciphertext/plaintext 與 tag，RSP FAIL case 還必須確認 plaintext/tag 皆不可觀察。

10.1.5 可重現的執行入口

Dynamic verification 在 Windows Git Bash 執行：

```
cd /c/project/FPGA3
bash verification/scripts/run_smoke.sh
bash verification/scripts/run_nist.sh
bash verification/scripts/run_uvm_xsim.sh
bash verification/scripts/run_uvm_regression.sh
bash verification/scripts/run_board_reset_xsim.sh
bash verification/scripts/run_formal.sh
bash verification/scripts/run_all.sh
```

Fresh board implementation 與程式化使用 Vivado 2021.1：

```
vivado.bat -mode batch \
  -source engrenring/synth_1g/board/\
run_arty_a7_100t_uart_rsp.tcl \
```

```
-tclargs <unique_output_label>

vivado.bat -mode batch \
  -source verification/board/program_board.tcl \
  -tclargs <exact_bitstream.bit>
```

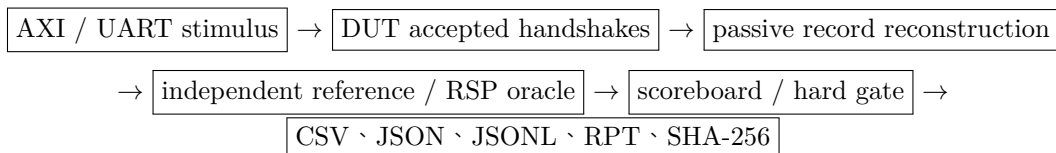
Hardware NIST 使用：

```
python engrenring/synth_1g/board/run_nist_rsp_uart.py \
  --port COM4 --baud 115200 --stop-on-fail \
  --bitstream <exact_bitstream.bit> \
  --output <evidence.jsonl>
```

最終 machine-readable 結論必須保留邊界：static implementation、dynamic simulation/UVM/NIST 與 hardware tests 為 PASS；formal 僅 19 個 combinational proofs，因此 integrated status 仍為 PARTIAL PASS，而不是任何 simulation 或 hardware suite failure。

10.2 Verification plan、oracle 與 comparison chain

驗證不以「模擬跑完」作為 PASS，而是先把需求轉成可觀測 stimulus、獨立 expected value、checker 與可保存證據。資料流如下：



UVM checker 不直接使用 sequence 或 driver 內的 expected transaction。passive_record_reconstruction_monitor 從實際 AXI-Lite AW/W handshakes、AXI-Stream input handshakes，以及 data/tag/result output handshakes 重建完整 record，再送進 scoreboard。Driver 與 checker 不共用同一 transaction object，可降低 stimulus 與 checker 同源錯誤；orphan output、缺少 result 或 timeout 都是 checker failure。

Golden model 位於 verification/uvm/aesgcm_reference_model_pkg.sv。AES S-box 由 $GF(2^8)$ 反元素與 affine transform 計算，不重用 DUT lookup table；GHASH 採 NIST SP 800-38D 的 128-step bit-serial multiplication，不重用 DUT 16-bit digit-serial datapath。Model 先以 FIPS-197 AES 與 SP 800-38D GCM 範例自我核對，再作為 UVM transaction-level oracle。

Scoreboard 對 encryption 比對完整 output payload、authentication tag 與唯一 result byte，且 success result 必須為 0x00；對 authentication-success decrypt 比對完整 plaintext，並驗證 success predicate。對 authentication-failure decrypt，UVM 要求不得釋放 plaintext，並檢查 (result == 0x01) == auth_ok；目前沒有強制失敗碼必須精確等於 0xFF。精確 0xFF failure status、空 data 與空 tag 是 FPGA NIST host runner 的另一個嚴格 checker boundary，不可誤稱由 UVM 完成。

FPGA NIST 不使用 UVM model 當 oracle，而直接使用六份 NIST .rsp 的 expected status、ciphertext/plaintext 與 tag。Encrypt 必須回傳 0x00、exact ciphertext 與 exact tag；successful decrypt 必須回傳 0x01 與 exact plaintext；RSP FAIL 必須回傳 0xFF，而且 data/tag 都為空。

JSONL reconciliation 不重新計算 AES-GCM ; 它獨立檢查 47,250 行、indices 0–47,249 連續唯一、六個來源各 7,875 筆、分類數與 mismatch counters , 避免只相信 aggregate PASS marker 。

表 10.6: Requirement-to-checker traceability matrix

ID	驗證需求	Oracle / checker 與 PASS criteria	Final result
VREQ-01	AES-GCM 功能正確	Independent SV model 與六份 NIST RSP ; payload/tag/status 依各 checker boundary 精確比對	UVM 24/0 ; simulation 與 FPGA NIST 各 47,250/0
VREQ-02	Auth-fail 不洩漏 plain-text	UVM bad-tag 要求 unauthenticated data 為空 ; board runner 另要求 0xFF、空 data/tag	11,908 expected failures ; board violations 0
VREQ-03	AXI record 完整	Passive handshake reconstruction ; exactly one result · 無 orphan output、timeout、UVM E/F	UVM E/F 0/0
VREQ-04	六模式 > 1 Gbit/s	100 筆 × 1024-bit throughput TB · 依 payload 與 cycle count 計算	accepted 1.065398 / 1.051705 Gbit/s
VREQ-05	Reset 短脈衝不可誤釋放	Directed reset TB 加 routed CDC/structural gate ; 1–3 sample pulse 與 chatter 全部拒絕	PASS ; release/assert edges 6/3
VREQ-06	Setup/hold/pulse closure	Vivado strict routed audit ; WNS/WHS/WPWS ≥ 0 · TNS/THS/TPWS/FEP = 0	PASS
VREQ-07	Route/DRC/CDC legality	Fully-routed、DRC、methodology、CDC schema、 <code>check_timing</code> hard gates	17,758/17,758 ; unsafe/E/CW 0
VREQ-08	實板 payload identity	SHA-256 比對 hardware-tested v8a 與 v8c raw configuration image	strict-audit .bin byte-identical · 8A19...B4332F3
VREQ-09	Algebraic formal correctness	Independent algebraic references + Yosys SAT ; 所有目標 assertions universal PASS	19/0 ; 只屬 PARTIAL PASS
VREQ-10	Standalone reuse	PartPin Approved identity-matched map 與 exact parent model · strict replay 必須 PASS	NO / NOT RUN

10.3 RTL regressions

Promotion 版本執行的主要驗證包括：

- AXI smoke : 7,676 cycles · PASS ;
- throughput : 六模式 16,820/17,039 cycles · 全部 > 1 Gbit/s ;
- Round48、Round49、Round53 security/directed regressions ;
- fast-zeroize 與 sensitive data-buffer clearing ;
- NIST 525 與 NIST 5,255 suites ;
- full NIST simulation : 47,250/47,250 · 17,740,527 simulation cycles 。

10.4 SVA : cycle-by-cycle temporal checker

本設計使用 SystemVerilog Assertions (SVA) 檢查「某個 clock edge 發生事件後，當下或下一個 edge 必須成立的局部時序規則」。SVA checker 放在 `verification/sva/`，由 `bind_all.sv` 依 module type 綁入 `ghash16`、`aes_key_context`、`aes_gcm_stream_core` 與 `aes_gcm_axi_top`；production RTL 不需插入 checker，也不把 SVA 納入 synthesis/bitstream。checker 可以觀察 DUT 內部 state、queue count、round keys 與 output handshake，因此比只看 transaction 最終結果更早指出違反規則的 cycle 與 property 名稱。

表 10.7: SVA checker 分組與檢查目的

Checker	Source statements	主要時序不變量
GHASH16 datapath/control	15	reset、H 必須先載入、busy/start、8 個 16-bit digits 的 T^{16} 位移、digit product、累加/final result、最小 request interval
AES key context	7	key expansion state transitions、S-box input/capture、ready/busy exclusivity、AES-128/192/256 的 10/12/14 rounds、ZEROIZE 後 1,920-bit round-key scrub
GHASH queue/control	5	blocked queue 穩定、valid/ready 才能 start、busy 不得 pop/start、ZEROIZE 或 queue-clear flush
Tag serializer	9	byte 順序、index/remaining count、fire 才前進、final-byte TLAST、backpressure 下 payload 穩定
AXI output stability	1	<code>valid && !ready</code> 時 valid 與 payload 必須保持；同一泛用 property 分別 bind 到 data/tag/result 三條輸出
Total	37	這個數字是 source-level <code>assert property</code> statements；目前 production top 展開為 39 個 active instances，並非 coverage 百分比

Property 以正緣 clock 取樣。`|->` 用來檢查同一取樣 edge 的 implication，`|=>` 檢查下一個 edge；`\$past(...)` 比對前一拍資料，`\$stable(...)` 檢查 backpressure 或 blocked queue 期間不得改變。依 property 使用 `disable iff (!rst_n)`，並在適用處加入 ZEROIZE 或 logical clear，使 reset/scrub/flush 的無效觀察窗不造成誤報；相對地，ZEROIZE 本身需要產生的 key/queue 清除效果由獨立 assertions 明確檢查。

GHASH checker 沒有呼叫 DUT 的直接 T^{POWER} 邏輯。它以 NIST shift/reduce 的 one-bit reference 重算 T^{16} 、16-bit digit product 與完整 $GF(2^{128})$ multiplication recurrence，再逐拍比對

digit-serial DUT；這可避免 checker 與 DUT 使用完全相同實作而一起犯錯。AXI 與 tag assertions 則針對 ready/valid protocol、TLAST 與 backpressure 保持性做 cycle-accurate 檢查。

執行時，`verification/scripts/common.sh` 先列出 production RTL 與六個 SVA/bind sources；`run_smoke.sh` 依序呼叫 `xvlog -sv`、`xelab`、`xsim`，跑 GHASH standalone 與 AXI integration。Runner 只有在 GHASH16_PASS tests=66、AXI_SMOKE_PASS cycles=7676 都出現，且 log 完全沒有 Assertion failed 時才回傳 PASS。本輪 retained integrated replay 的 smoke 為 0 assertion failures。run_nist.sh 也編入同一組 SVA，完整 simulation 為 47,250/47,250、fail 0、17,740,527 cycles，且同樣以任何 assertion failure 作 hard fail。

SVA、UVM 與 formal 的責任不同：UVM 由 transactions、independent golden model 與 scoreboard 檢查 end-to-end data/tag/status；SVA 在這些 dynamic simulation traces 上逐拍檢查局部 control/protocol invariants；Yosys SAT 的 19 個 combinational assertions 是另一組 formal harness，並不是這 37 條 SVA 的 exhaustive proof。目前 SVA 來源內沒有 cover property 或 assume property，也未收集 property activation、vacuity 或 assertion coverage，所以只能宣稱「retained tests 中 0 assertion failures」，不能宣稱 37/37 都被觸發或 100% assertion coverage。它也不是 post-route、實板、sequential whole-design、liveness、CDC、timing、power 或 side-channel formal proof。

證據 provenance 另有一個已知缺口：retained `verification/results/sva/sva_summary.md` 仍記錄舊 RTL SHA，因此 current 判定採用 promotion manifest 綁定的 integrated `run_all.log` 與 retained smoke/NIST logs；現有 SVA runner 尚未像 UVM runner 一樣保存每個 compile input 的 SHA-256。後續應補上 compile-input manifest 與 activation/vacuity coverage，再提升成 exact-source SVA attestation。

SVA 證據台帳：原始 checker、bind、執行 gate 與 retained trace

SVA 的證據不是單一截圖，而是一條可回溯的 chain：**property source** 定義要檢查的規則，**bind source** 證明它接到 production RTL，**runner** 定義失敗 gate，最後由 **XSim logs** 保存實際執行結果。表 10.8 列出本報告採用的 retained evidence；報告答辯時可依此順序開啟檔案，不需要以口頭敘述取代原始紀錄。

表 10.8: SVA evidence ledger 與可宣稱的範圍

證據層	原始檔 / log 位置	可直接確認的事實與邊界
Property source	<code>verification/sva/ghash16_sva.sv</code> 、 <code>aes_key_context_sva.sv</code> 、 <code>ghash_queue_sva.sv</code> 、 <code>aesgcm_tag_sva.sv</code> 、 <code>axis_output_sva.sv</code>	37 個 source-level <code>assert property</code> statements，分別定義 GHASH、key scrub、queue、tag 與 AXI output stability 的逐拍規則。這證明「寫了哪些 property」，不是已觸發或已證明。
Bind / elaboration target	<code>verification/sva/bind_all.sv</code>	<code>bind</code> 把 checker 接到 <code>ghash16</code> 、 <code>aes_key_context</code> 、 <code>aes_gcm_stream_core</code> 與 <code>aes_gcm_axi_top</code> ；其中 <code>axis_output_sva</code> 對 data/tag/result 三條輸出各 bind 一次，production top 共展開 39 個 active instances。

證據層	原始檔 / log 位置	可直接確認的事實與邊界
Hard-fail runner	verification/scripts/run_smoke.sh、 verification/scripts/run_nist.sh	兩個 runner 都將 RTL 與 SVA sources 一起傳入 <code>xvlog -sv</code> ，再經 <code>xelab/xsim</code> 執行；若 log 出現 <code>Assertion failed</code> 即以 <code>non-zero exit</code> 結束。Smoke 另要求 <code>GHASH16_PASS tests=66</code> 與 <code>AXI_SMOKE_PASS cycles=7676</code> ；NIST 另要求 <code>NIST_ALL_PASS 47250/47250</code> 。
Smoke execution trace	verification/results/smoke/run_smoke.log； verification/results/run_all.log	retained log 記錄 <code>GHASH16_PASS tests=66 digit_cycles=8 external_ii=9、AXI_SMOKE_PASS cycles=7676</code> ；integrated summary 記錄 <code>suite=smoke status=PASS assertion_failures=0</code> 。這是兩個 smoke trace 內未違反 assertion 的證據。
Full NIST execution trace	verification/results/nist/run_nist.log； verification/results/run_all.log	log 顯示 SVA checker modules 隨 <code>tb_nist</code> 編譯，並記錄 <code>NIST_TOTAL_SUMMARY pass=47250 fail=0 total=47250</code> 與 <code>NIST_ALL_PASS 47250/47250</code> ；integrated summary 記錄 <code>suite=nist status=PASS</code> 。這是完整 NIST dynamic trace 的 SVA hard-gate execution evidence，但不是 assertion activation/vacuity coverage。

判讀規則：上述 logs 支持「retained smoke 與 NIST execution 中沒有 assertion failure」；不支持「每條 assertion 都至少觸發一次」、「100% assertion coverage」或「對 current exact RTL 的 cryptographic attestation」。最後一項仍受限於 SVA runner 未保存 compile-input SHA-256，且舊 `sva_summary.md` 的 RTL SHA 不可單獨當作 current-source provenance。

10.5 UVM、coverage、formal 與 integrated replay

XSim 2021.1 的 UVM 1.2 constrained-random regression 使用 seeds 175001、175019 與 175039，每個 seed 8 個 scenarios，總計 scoreboard 24 checked、0 failed，UVM error/fatal 為 0/0，每個 seed 的維護中 functional coverage model 為 100.00%。Coverage bins 包含 AES-128/192/256、encrypt/decrypt、96/non-96-bit IV、empty/short/one-block/multi-block AAD/data、valid/invalid authentication，以及 `key×mode`、`mode×IV` 與 `mode×data crosses`。本輪另保存 15 個實際 compilation inputs 的 SHA-256，並把全域 UVM error/fatal counts 納入最終 PASS gate。

ModelSim ALTERA Starter 10.5b 以 UVM 1.2 deterministic mode 執行一筆固定 AES-128 record，scoreboard 1/0、UVM error/fatal 0/0。Starter license 不支援本 regression 所需的 constrained randomization 與 covergroups，因此 ModelSim constrained-random 明確標為 NOT RUN、functional coverage 標為 NOT COLLECTED；三-seed random 與 functional coverage 證據來自 XSim，不從 deterministic smoke 推論。

Yosys 0.33 SAT 完成 19 個 assumption-free combinational assertions：GHASH $T^{0...16}$ 對 independent NIST one-bit shift reference 為 17/0，AES S-box 對 algebraic $GF(2^8)$ inverse/affine reference 與 injectivity 為 2/0。這是 **partial formal PASS**，不包含 sequential GHASH control、FIFO/ZEROIZE temporal properties、whole AES-GCM equivalence、liveness、CDC、timing、power 或 side-channel proof，因此不宣稱 full formal closure。

Source/verification promotion commit 4246ebc7a72ce8eb1a49d56969aae6e151578c11 的 integrated run_all 結果是 5 PASS、1 PARTIAL PASS、0 NOT RUN、0 FAIL，共 6 suites；唯一 partial suite 是上述 combinational formal subset，不是 simulation failure。整體 machine-readable 狀態為：

```
175MHZ_STATIC_SIGNOFF_SIMULATION_UVM_RESET_AND_HARDWARE_NIST_PASS_FORMAL_SUBSET_PARTIAL
```

10.6 Positive、negative tests 與未覆蓋範圍

Positive tests 包含 AES-128/192/256 encryption/decryption、96-bit 與 non-96-bit IV、empty/short/one-block/multi-block AAD/data、valid authentication、stable run request 在第 6 個 sampled edge 釋放、六模式 throughput、FPGA smoke 6/0，以及 NIST encrypt/decrypt-success。

Negative tests 包含每個 UVM seed 的 AES-256 corrupted-tag decrypt、FPGA NIST 的 11,908 筆 expected authentication failures、unauthenticated plaintext 不得輸出、button 與 MMCM LOCKED 的 1/2/3-sample pulses、alternating reset-request chatter、invalid request 在第 3 個 sampled edge 使 distributed reset 生效，以及 Round48/49/53 與 FIFO clear/refill directed regression 8/8。

目前三-seed UVM regression 尚未執行 random output backpressure、ZEROIZE phase injection、descriptor queue saturation、early/late TLAST fault injection；這些模式被約束為 BP_NONE、ZP_NONE、FAULT_NONE，不得因 sequence class 已宣告就計入 coverage。另未收集 code coverage 或 assertion coverage；SVA smoke 的 37 properties、0 assertion failures 是執行結果，不是 assertion-coverage 百分比。

10.7 FPGA programming 狀態

Vivado Hardware Manager 透過 target Digilent/210319BE76DEA 找到 xc7a100t_0，program_board.tcl 回報 BOARD_PROGRAM_SUMMARY status=PASS。程式化的 v8a bitstream SHA-256 為

```
22791C17939C1BBC43539C35ABB19C31
896425D26AA4721D5E4D4D3ACC5A686E.
```

該 Tcl 會呼叫 program_hw_devices 與 refresh_hw_device，但不產生 configuration-memory readback attestation；因此本報告不額外宣稱 EOS/DONE/CRC 狀態。後續六模式 UART smoke 的 summary 全部綁定同一 bitstream hash，為板上 operational evidence。

Final strict audit v8c 另輸出 SHA-256 為 DC951AAE...0934942 的 .bit；它的完整檔案 SHA-256 與板測 v8a .bit 不同，所以本報告不宣稱 v8c .bit 曾被程式化或與 v8a 檔案相同。兩者的 raw configuration .bin SHA-256 則同為 8A19D8E5...B4332F3；這是 static-audit output 與 hardware-tested configuration 之間的 exact binary identity boundary，完整 hashes 見附錄 A.1。

10.8 六模式 smoke 與 47,250 筆 NIST RSP final result

表 10.9: FPGA hardware verification final status

Test	Executed	Fail	Status / evidence
AES-128 encrypt smoke	1	0	PASS ; exact v8a bit hash
AES-192 encrypt smoke	1	0	PASS ; exact v8a bit hash
AES-256 encrypt smoke	1	0	PASS ; exact v8a bit hash
AES-128 decrypt smoke	1	0	PASS ; exact v8a bit hash
AES-192 decrypt smoke	1	0	PASS ; exact v8a bit hash
AES-256 decrypt smoke	1	0	PASS ; exact v8a bit hash
Full NIST GCM RSP	47,250	0	PASS ; 47,250/47,250 · first failure null

實板 runner 使用 COM4、115200 baud，逐筆比對 status、ciphertext/plaintext 與 authentication tag；decrypt FAIL vectors 必須回傳 AUTH_FAIL 且不釋放 plaintext。Final summary 為 requested=executed=pass=47250、fail=0、first_failure=null。JSONL 恰有 47,250 行、global indices 0–47,249 連續且唯一；六個 RSP source 各 7,875 筆。分類為 encrypt 23,625、decrypt 23,625，其中 decrypt success 11,717、expected authentication failure 11,908。Status/data/-tag mismatch 均為 0，且第二位獨立 reconciliation 亦通過。

Host-observed per-case latency 的 min/mean/median/p95/p99/max 分別為 9.101、25.376、31.440、32.289、47.797、49.779 ms；全量 elapsed time 為 1,208.505 s，平均 39.098 cases/s。這組 latency 包含 UART 與 host transaction overhead，只能解讀為 board-validation harness 表現。

FPGA NIST RSP Source Coverage		
47,250 pass; 0 fail		
RSP source	Cases	Result
gcmEncryptExtIV128	7,875	PASS
gcmEncryptExtIV192	7,875	PASS
gcmEncryptExtIV256	7,875	PASS
gcmDecrypt128	7,875	PASS
gcmDecrypt192	7,875	PASS
gcmDecrypt256	7,875	PASS

Host-Observed Latency	
COM4, 115200 baud; not core latency	
Statistic	ms / case
Minimum	9.101
Mean	25.376
Median	31.440
P95	32.289
P99	47.797
Maximum	49.779
Bitstream 22791C...5A686E JSONL 56DA5E...0C59761	

图 10.1: Full FPGA NIST source coverage 與 host-observed latency。左圖六個 RSP source 各執行並通過 7,875 筆；右圖呈現 min、mean、median、p95、p99 與 max。這不是 AES-GCM core cycle latency。

第十一章 遇到的問題、根因與解法

表 11.1: Verification failure triage：症狀、修正、oracle 與證據

可觀測症狀 / 風險	根因與修正	驗證方式 / Oracle	結果與證據
175 MHz 原架構 throughput < 1 Gbit/s	First-block startup bubble；加入兩 個 first-block engines、16- entry keystore FIFO 與兩 筆 sensitive queue	keystream 100 筆 × 1024-bit、六模式 accepted-cycle measurement	16,820/17,039 cycles； 1.065398/1.051705 Gbit/s
Board v3 synth WNS -1.192 ns、 TNS -1506.136 ns、FEP 1489	1024-bit UART FF arrays 形 成 wide mux/high fanout；改 為 8-bit byte memories	v3/v4/v8 utilization 與 timing re- port 差分	LUT 減 10,048、FF 減 6,434；final WNS +0.029 ns
Protocol simula- tion stream byte 不前進	Helper function 隱含讀 global registers · XSIM 未依預期觸發；改為直接 combinational expression	六模式 protocol simulation 與 ex- act output compare	6/6 PASS
Placed setup pass 但 hold 9 end- points fail	Setup phys-opt 產生 short hold paths；採 Explore route 與 post-route hold repair	Post-route setup/hold/pulse audit	WHS +0.044 ns、 THS/FEP 0
cdc_final_v5 向 量通過但 reset 仍 有漏洞	Pure four-FF shift 把 one- sample pulse 延後成 release sample；改為 2-stage sync + 4-stage qualifier	1/2/3-sample pulse、chatter、re- lease/assert edge directed TB	v5 不 promotion；新結 構 PASS 6/3 edges
resetqual_final_v6 降低四樣本門檻	只有 2-stage qualifier，且 full hardware NIST 未完成	Topology audit + required-run completion gate	不 promotion
v7/v8/v8b build/audit false rejection	Tcl boolean/list 與 hierarchi- cal VCC 名稱的 internal rep- resentation 比較錯誤；改做 object canonicalization	對同一 SHA-matched routed DCP 重跑 strict audit	v8c 對 A917...016B PASS
v8a 與 v8c .bit hash 不同	.bit container/header 不同，比對 不等於 configuration payload 不同	raw configuration .bin SHA- 256	.bin 完全相同；不宣稱 .bit 相同

可觀測症狀 / 風險	根因與修正	驗證方式 / Oracle	結果與證據
Auth-fail plaintext leakage 風險	Decrypt data 必須等 tag 證後才釋放 · clear/abort 須清 sensitive queue	UVM bad-tag、directed ROIZE/FIFO 與 11,908 個 FAIL cases	ZE- Unauthorized data 0 ; board status/data/tag violations 0
功率數字可信度有限	無代表性 SAIF/VCD 或實板 rail 量測	Vivado routed vectorless report 明示 confidence boundary	· 0.461 W、Medium confidence estimate
UART 無法顯示 1 Gbit/s	115200 baud、8N1 ceiling 92.16 kbit/s	分離 core cycle counter 與 host/UART wall time	39.098 cases/s 只屬 board harness
Legacy top / historical 200 MHz 不可作本版 production evidence	Legacy top flow stale/missing dependencies ; historical 200 MHz 是保留的 Git 歷史 · 與 current target 不同	Source/build audit 與維護中 175 MHz source list	175 Production 僅 175 MHz ; 200 MHz 未重跑
Standalone Part-Pin 無法 signoff	無 approved identity-matched map 與 parent timing model	run_partpin_replay.tcl strict gate	NO / NOT RUN

第十二章 工具、Tcl 驅動方式與檔案位置

12.1 本次使用的工具

表 12.1: 工具與用途

工具	版本 / 介面	用途
Vivado	2021.1 build 3247384	synth、opt、place、phys_opt、route、timing、utilization、power、DRC、bitgen、Hardware Manager
XSIM	Vivado 2021.1	smoke、throughput、security regressions、NIST simulation、UART protocol simulation
SVA	XSim 2021.1	37 source assertions (39 top-level active instances) ; 以 bind 檢查 GHASH、key/ZEROIZE、queue、tag 與三條 AXI outputs
UVM 1.2	XSim 2021.1	3 seeds、24 scenarios、scoreboard 與 maintained functional coverage
UVM 1.2	ModelSim Starter 10.5b	Deterministic smoke 1/0 ; Starter constrained random/coverage 未執行
Yosys SAT	0.33	GHASH shift transform 與 AES S-box 的 19 個 combinational assertions
Tcl	Vivado Tcl shell	可重現 flow、DCP hash gate、reports 與 hard gates
PowerShell	Windows	啟動 batch flow、整理驗證輸出與環境檢查
Python + pyserial	Host COM4	parse NIST .rsp、送 UART frames、比對結果、輸出 JSONL evidence
Get-FileHash	PowerShell	SHA-256 DCP/bitstream/evidence identity
Git	branch/commit	保存 RTL、tests、Tcl 與可追溯變更
Graphviz	14.1.3	由 DOT 可重現產生全部報告架構圖、表格式圖與 SVG/PNG
LaTeX/Tectonic	XeTeX-compatible	產生本報告 PDF

「OpenShell」不是此專案的必要工具。這次主要使用 PowerShell 呼叫 Vivado batch Tcl ; Vivado 自身也有 Tcl console。GUI 只用於 Hardware Manager/JTAG programming 與需要時的 interactive Device view。

12.2 如何驅動 Tcl

Fresh board build :

```
C:\Xilinx\Vivado\2021.1\bin\vivado.bat -mode batch \
-source C:\project\FPGA3\engrenring\synth_1g\board\run_arty_a7_100t_uart_rsp.tcl \
-tclargs <unique_output_directory>
```

從 verified synth DCP resume :

```
C:\Xilinx\Vivado\2021.1\bin\vivado.bat -mode batch \
-source C:\project\FPGA3\engrenring\synth_1g\board\run_arty_a7_100t_uart_rsp_from_synth
.tcl \
-tclargs <synth.dcp> <expected_sha256> <unique_output_directory>
```

Host NIST RSP runner :

```
python C:\project\FPGA3\engrenring\synth_1g\board\run_nist_rsp_uart.py \
--port COM4 --baud 115200 --stop-on-fail \
--bitstream <final.bit> --output <evidence.jsonl> <six_rsp_files>
```

SVA smoke、完整 NIST simulation 與 integrated replay (在 Windows Git Bash 執行):

```
cd /c/project/FPGA3
bash verification/scripts/run_smoke.sh
bash verification/scripts/run_nist.sh
bash verification/scripts/run_all.sh
```

run_smoke.sh 與 run_nist.sh 都把任何 Assertion failed 視為 non-zero failure ; run_all.sh 再把 smoke、NIST、UVM、reset 與 formal 結果彙整。為避免 Windows simulator path 問題，整套 runner 明確拒絕從 WSL 啟動。

12.3 RTL、SVA 與 wrapper 位置

表 12.2: 主要 RTL 與 verification source files

功能	絕對路徑
AXI top wrapper	C:/project/FPGA3/engrenring/rtl/source/aes_gcm_axi_top.v

功能	絕對路徑
AES-GCM stream core	C:/project/FPGA3/engrenring/rtl/source/stream_core/aes_gcm_stream_core.v
Main AES engine	C:/project/FPGA3/engrenring/rtl/source/stream_aes/aes_block_engine.v
First-block AES engine	C:/project/FPGA3/engrenring/rtl/source/stream_aes/aes_first_block_engine.v
Key expansion	C:/project/FPGA3/engrenring/rtl/source/stream_aes/aes_key_context.v
AES S-box	C:/project/FPGA3/engrenring/rtl/source/aes_core/sbox.v
GHASH16	C:/project/FPGA3/engrenring/rtl/source/stream_ghash/ghash16.v
Generic FIFO	C:/project/FPGA3/engrenring/rtl/source/stream_core/stream_fifo.v
AXI-Lite registers	C:/project/FPGA3/engrenring/rtl/source/stream_axi/axi_lite_regs.v
UART board top	C:/project/FPGA3/engrenring/rtl/source/board/artty_a7_100t_aes_gcm_uart_rsp_top.v
UART RSP bridge	C:/project/FPGA3/engrenring/rtl/source/board/aes_gcm_uart_rsp_bridge.v
UART RX/TX	C:/project/FPGA3/engrenring/rtl/source/board/uart_rx.v 、uart_tx.v
SVA checkers / bind	C:/project/FPGA3/verification/sva/*.sv 、bind_all.sv

12.4 Final implementation、圖片與證據位置

Final strict static-audit evidence directory :

C:
/project/FPGA3/engrenring/synth_1g/board/output/ece78ce_uart_rsp_175_resetqual4_audit_v8c

Hardware programming、smoke 與 NIST evidence directory 為

C:
/project/FPGA3/engrenring/synth_1g/board/output/ece78ce_uart_rsp_175_resetqual4_audit_v8a.

其 immutable synth/placed/routed DCP 來源目錄為

C:
/project/FPGA3/engrenring/synth_1g/board/output/ece78ce_uart_rsp_175_resetqual4_final_v8.

Vivado 原生圖片、vector schematics、互動式 RPX 與文字報告位於

C:/project/FPGA3/docs/report/figures/vivado_native.

其 manifest 綁定 synth/routed DCP SHA-256 ; export_vivado_native_reports.tcl 負責 batch 匯出，open_vivado_native_visuals.tcl 由同一 routed DCP 建立可重現的 GUI reports。

其中重要檔案如下：

檔案	用途
v8c arty_a7_100t_aes_gcm_uart_rsp_top.bit、 arty_a7_100t_aes_gcm_uart_rsp_top.bin	strict-audit outputs ; .bin 與 hardware-tested v8a config image exact match
v8a arty_a7_100t_aes_gcm_uart_rsp_top.bit	Hardware Manager 實際程式化，且由 smoke/NIST exact hash 綁定的 bitstream
v8 *_synth.dcp、 *_placed.dcp、 *_routed.dcp	fresh build checkpoints ; routed DCP 是 v8c strict audit 的 hash-gated input
v8c signoff_metrics.txt	routed DCP identity、setup/hold/pulse-width、route/CD-C/reset/check_timing hard-gate summary
v8c timing_routed.rpt	WNS/TNS/WHs/THs/WPWS/TPWS 與 critical paths
v8c utilization_routed.rpt	LUT/FF/LUTRAM/BRAM hierarchy
v8c route_status.rpt	fully routed nets 與 routing error
v8c drc_routed.rpt、 methodology_routed.rpt	DRC 0 ; methodology E/CW 0 與 2 個一般 warnings
v8c cdc_routed.rpt、 check_timing_routed.rpt	synchronizer schema 與 exact false-path allowlists
v8c power_vectorless_routed.rpt	0.461 W Medium-confidence vectorless estimate
v8c clock_utilization_routed.rpt	MMCM/BUFG、clock loads 與 clock regions
vivado_native/*.png	Device placement、routing、timing、clock、utilization、power、congestion、package 與 DRC 原生 GUI 圖
vivado_native/*.svg、 *.pdf	Vivado write_schematic 原生 synthesis / routed critical-path 向量圖

檔案	用途
vivado_native/*.rpx 、 *.rpt	可在 Vivado 重開的互動式 reports 與對應文字證據
v8a program_board.log	Hardware Manager programming terminal evidence
v8a hw_smoke_*.jsonl 、 hw_smoke_*.summary.json	AES-128/192/256 encrypt/decrypt 六模式 · 6/0
v8a hw_nist_47250.jsonl	47,250 行逐向量 evidence ; indices/source/classification/status- data-tag reconciliation 全通過
v8a hw_nist_47250. summary.json 、 hw_nist_47250_run.log	47,250/47,250 、fail 0 、first failure null 、elapsed 1,208.505 s

本報告的 Graphviz 圖檔與 DOT sources 位於 C:/project/FPGA3/docs/report/figures ; C:/project/FPGA3/docs/report/generate_figures.py 僅接受 Graphviz 14.1.3 。

Vivado 原生圖位於 vivado_native 子目錄。這些圖由 Vivado 2021.1 checkpoint 直接匯出或擷取；未使用 Graphviz 、matplotlib 或 image generation 重畫。

第十三章 結論與後續工作

本版已完成可燒錄的 Arty A7-100T AES-GCM 實作，175 MHz v8c strict static audit 的 setup、hold、pulse-width、route、DRC、methodology E/CW、CDC/reset schema 與 `check\_timing` hard gates 全部通過。Hardware-tested v8a bitstream 的 configuration .bin 與 v8c strict-audit .bin 完全相同；六模式 smoke 為 6/0，full hardware NIST 為 47,250/47,250、fail 0，且第二次獨立 reconciliation 通過。核心透過兩個 first-block engines、三 context shared AES pipeline、16-entry keystream FIFO、兩筆 sensitive data queue 與 16-bit digit-serial GHASH，在單一 8-bit stream 下達到 1.065398 Gbit/s encryption 與 1.051705 Gbit/s decryption。

驗證結論必須保持分層：static implementation、dynamic simulation/UVM/NIST 與實板驗證都是 PASS；integrated overall 是 **PARTIAL PASS**，唯一原因是 formal 僅涵蓋 19 個 combinational assertions，並非任何 suite failure 或 NOT RUN。UVM、FPGA NIST 與 routed audit 使用不同 oracle/checker，互相補強但不互相取代。

Standalone OOC PartPin 狀態仍為 **NO / NOT RUN**：目前沒有 approved、identity-matched production HD.PARTPIN_LOCS map 與 parent timing model。此限制與已通過的 full-board route 是不同 signoff boundary。

下一步建議依風險排序：

1. 封存 v8c strict-audit 與 v8a board evidence 的 manifest、SHA-256 與 reconciliation 規則；未來重跑必須先核對 routed DCP 與 configuration image identity。
2. 以代表性 SAIF/VCD 重跑 power，或量測 FPGA 電源 rails，替換 Medium-confidence vectorless 數字。
3. 擴充 sequential formal adapter/solver，覆蓋 GHASH control、FIFO/ZEROIZE temporal behavior 與 top-level liveness；不把 19 個 combinational assertions 誤當 full closure。
4. 若交付範圍需要 PartPin，先取得 approved identity-matched production map 與 parent timing model，再執行 strict replay。
5. 若要證明板上 crypto datapath 的 1 Gbit/s，而不只是 simulation，加入 on-chip cycle counter + BRAM traffic generator，或改用 Ethernet/PCIe/高速 AXI DMA。
6. 若要正式交付 Vivado Block Design，將 `aes_gcm_axi_top` 另行 package 成 IP-XACT，並建立 CDC、register-map 與 driver 文件。

附录 A Artifact identity 與 hashes

表 A.1: Final artifact SHA-256

Artifact	SHA-256
v8 synth DCP	C5D9858BDD097622C7AA76A1843CB3D4 9EF7646470BC4ACDAF9BDB27882914E6
v8 placed DCP	82BAA9B402F825B3127202255F5787BA 82A8E7F34291A0DDA5F4C86689C4119D
v8 routed DCP / v8c strict-audit input	A91758ED87EB56648EEF0CAB67B2C50D 198D00A2D5088E624B507687D99C016B
v8c audit script	62C653E5E49723A51726B92B0D475BA1 1C55F24A3648D59359A2ABCE99C9286E
v8c bitstream	DC951AAE0D388DA54836AD5950CA87CB 8EEE4A94E5E7EFBB1A685439F0934942
v8c binary / hardware-tested v8a binary	8A19D8E5900063B64922DC9B9AD323F9 53386367CA1D4FA1A61935543B4332F3
Hardware-tested v8a bitstream	22791C17939C1BBC43539C35ABB19C31 896425D26AA4721D5E4D4D3ACC5A686E
v8c signoff metrics	CBC7D4F16D9D54A67CC6B10E4A9DC612 888112933D6263F3F2D69932FB0946CB
v8c routed power report	687014688995046BACE3FFA09CB4E3E3 B76A551B0F9A1D3C92C7C6874A9EF351
UVM summary CSV	F46216BF196C08ED466200EE34E3F2A7 B626EB376E7C0F8A758DCB47B3225641
UVM summary JSON	F5B8E59504D967B6841AD73E0B6E3D8 361162E251CF7FED2F303682C8B8500AF
UVM compile-input manifest	D36DA381A5326EEBB1DABDA4AD63D45 421ACE5E433CE38A3EF80494002233292
UVM aggregate run log	6AC1D83E3C90615448F6391350707963 D544F4D42CB7FC6205A7A0C3B7189EC5
ModelSim deterministic UVM log	01EF5B244A0C6C8FA8FD812E7C7F743 7E3EE017CEF67D43EF5A46E783404D0DE
Formal run log	C1FB4DA19869532E1831069A635BF71F 7C6714237DD5C08C7EDF6A16CCBC0C96

Artifact	SHA-256
Reset directed run log	D01050A0A6592BB549F1978A123B4C669 A326A9AE6F882B57FEE989E7B3C1879
Reset directed simulation log	78106EEE91399C6849721C53949B3DBA7 D9D1ED1A3F0831A080C87E90A5F0A68
Integrated <code>run_all</code> log	14F4244DE577CF1B1D80B7B2A9FBE1E6 B33A07AF5629341A7F523DFA3DB8D7CA
Full simulated NIST log	5EDC8CEE2AFF8D05D6538B8CFE911D8B7 75FFECED758FCBFD36786D9045BBCE3
NIST FPGA JSONL	56DA5E1B7110AE8A5622B5847211A842 C4D2A2A22A08EE02AA7D32EFB0C59761
NIST FPGA summary	5DF2EA729BBD0C19C6677AFD13F73DD4 F04BB53694B9552A3FF4D9D464604D06
NIST FPGA run log	B861E8500B214CB85A98A1FDC231C3E0 97DDA60F237EA3A49021E6BCECEFEEB4

附錄 B 參考資料

- [1] National Institute of Standards and Technology, *Advanced Encryption Standard (AES)*, FIPS PUB 197, 2001. Local file: `reference/[02]NIST_FIPS_197_AES.pdf`.
- [2] Morris Dworkin, *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*, NIST SP 800-38D, 2007. Local file: `reference/[03]NIST_SP_800-38D_GCM_GMAC.pdf`.
- [3] Akashi Satoh, “High-Speed Hardware Architectures for Authenticated Encryption Mode GCM,” ISCAS, 2006. Local file: `reference/[08]Satoh_High-Speed_GCM_Hardware_Architectures.pdf`.
- [4] Luca Henzen and Wolfgang Fichtner, “FPGA Parallel-Pipelined AES-GCM Core for 100G Ethernet Applications,” 2010. Local file: `reference/[09]Henzen_Fichtner_FPGA_Parallel_Pipelined_AES-GCM_100G.pdf`.
- [5] Arundhati Joshi, P. K. Dakhole, and Ajay Thatere, “Implementation of S-Box for Advanced Encryption Standard,” ICETECH, 2015. Local file: `reference/[17]Joshi_Implementation_of_S-Box_for_AES.pdf`.
- [6] Xinmiao Zhang and Keshab K. Parhi, “High-Speed VLSI Architectures for the AES Algorithm,” *IEEE Transactions on VLSI Systems*, vol. 12, no. 9, 2004. Local file: `reference/[18]Zhang_Parhi_High-Speed_VLSI_AES.pdf`.
- [7] Akashi Satoh, Sumio Morioka, Kohji Takano, and Seiji Munetoh, “A Compact Rijndael Hardware Architecture with S-Box Optimization,” ASIACRYPT, 2001. Local file: `reference/[20]Satoh_Compact_Rijndael_Hardware_Architecture.pdf`.
- [8] Gang Zhou, Harald Michalik, and László Hinsenkamp, “Improving Throughput of AES-GCM with Pipelined Karatsuba Multipliers on FPGAs,” ARC, 2009. Local file: `reference/[28]Zhou_Pipelined_Karatsuba_AES-GCM_FPGA.pdf`.